Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3069 Computación Paralela y Distribuida
Ing. Juan Luis García

Proyecto 2
Fuerza bruta sobre DES con Open MPI

Esteban Cárcamo	23016
Nicolás Concuá	23197
Diego López	23747

Repositorio: https://github.com/nicoCT04/Proyecto2-Paralela

Guatemala, 14 de octubre de 2026

Índice

Introducción 4
Objetivos 4
Programas desarrollados 4
Estructura del informe 5

Parte A.1 a A.3: DES y la búsqueda secuencial 6
A.1 ¿Qué es DES? 6
A.2 Diagrama de flujo 7
A.3 Búsqueda secuencial con distintas longitudes de llave 8

Parte A.4 a A.6: Rutinas de la base y primitivas MPI 10
A.4 Por qué no compila rpc/des_crypt.h 10
A.5 Las rutinas de la base 10
A.6 Primitivas MPI del aviso de llave 12

Parte B.1 y B.2: Pruebas del cifrado y del naive 15
B.1 Cifrar y descifrar desde un .txt 15
B.2 Naive con 4 procesos contra el secuencial 15

Enfoques de paralelización 20
Esqueleto común 20
Naive: bloques contiguos 21
Intercalado 21
Bloque-cíclico 22
Maestro-trabajador 22
Híbrido MPI + OpenMP 24
Resumen teórico 25

Análisis de speedup y del comportamiento según la llave 27
Lo teórico frente a lo medido 27
El naive depende de la llave y su superlineal es aparente 28
La llave 60 000 000: por qué fue un mal ejemplo para comparar 29
Fugas de eficiencia 32
Barrido de procesos, Amdahl y Gustafson 32
El híbrido frente a la versión sin OpenMP 34

Conclusiones y recomendaciones 35
Conclusiones 35
Retos y cómo se resolvieron 35
Aprendizajes 36
Recomendaciones 36

Referencias **37**

Anexo 1: Catálogo de funciones y librerías **38**

des_utils.c: DES sobre OpenSSL 38

options.c: Parámetros 38

problem.c: Problema y reporte 39

cifrar.c y secuencial.c: Partes B.1 y A.3 40

search_mpi.c: Módulo común MPI 40

Programas MPI e híbrido 42

scripts/analizar.py: Análisis de la campaña 43

scripts/key_position_plots.py: Figuras 12 y 13 44

visualizer/: Visualizador web 45

Librerías 45

Anexo 2: Bitácora de pruebas y speedups **48**

Máquina y software 48

Procedimiento 48

Verificación 49

Campaña: tiempos (mediana de 5, en segundos) 49

Capturas 49

Introducción

El *Data Encryption Standard* (DES) protege un texto con una llave de 56 bits. Hoy esa llave es demasiado corta: probar las $2^{56} \approx 7.2 \cdot 10^{16}$ combinaciones está al alcance de una máquina dedicada (Electronic Frontier Foundation [EFF], 1998). Por eso el Instituto Nacional de Estándares y Tecnología de Estados Unidos retiró el estándar en 2005 (National Institute of Standards and Technology [NIST], 2005).

Ese ataque, la fuerza bruta, es un buen problema para estudiar la programación paralela. Cada llave se prueba de forma independiente, así que el trabajo se reparte casi sin comunicación. Sin embargo, la forma de repartirlo decide si el speedup es excelente o nulo.

En este proyecto se cifra un texto con DES y una llave privada. Después se recupera la llave probando combinaciones hasta que el texto descifrado contiene una frase clave conocida, «es una prueba de». La búsqueda está escrita en C, con OpenSSL para DES y Open MPI para repartir el trabajo entre procesos (Message Passing Interface Forum [MPI Forum], 2023). Como extra, OpenMP reparte el trabajo de cada proceso entre hilos (OpenMP Architecture Review Board [OpenMP ARB], 2021).

Objetivos

El objetivo general es implementar y modificar un programa paralelo con MPI en C que descifre por fuerza bruta un texto cifrado con DES. Primero se resuelve de forma secuencial y después con el reparto naive del programa base del curso.

Sobre esa base se proponen enfoques alternativos de reparto. Cada uno se mide con las métricas del curso: speedup, eficiencia, costo y efectividad, junto con las leyes de Amdahl (1967) y Gustafson (1988).

El último objetivo es explicar por qué el naive se comporta de forma inconsistente según la llave. Con esa explicación se demuestra que el proyecto no es superlineal: los speedups mayores que p que aparecen son un efecto de búsqueda.

Programas desarrollados

La tabla resume los siete programas del proyecto y el capítulo donde se discute cada uno.

Programa	Qué hace	Capítulo
`cifrar`	Cifra y descifra un `.txt` con una llave (Parte B.1)	Parte A y pruebas B
`secuencial`	Fuerza bruta en un solo proceso (Parte A.3); da el T_s	Parte A
`naive`	Bloques contiguos, uno por proceso (Parte B.2)	Pruebas B y enfoques
`intercalado`	El proceso r prueba r, r + p, r + 2p, …	Enfoques
`ciclico`	Chunks de 2^c llaves en round-robin	Enfoques

Programa	Qué hace	Capítulo
`maestro_trabajador`	El proceso 0 reparte chunks bajo demanda	Enfoques
`hibrido`	Intercalado con p procesos MPI y T hilos de OpenMP (extra)	Enfoques

Todos comparten el arranque, el aviso de llave y el reporte del módulo `search_mpi`, y escriben una línea CSV con el mismo formato. Tres scripts completan el trabajo: `verificar.sh` comprueba que cada programa halla la llave real, `campana.sh` corre las mediciones y `analizar.py` genera las tablas y gráficas a partir de la mediana. Un visualizador web (`visualizer/`) corre los programas reales y dibuja qué llaves prueba cada proceso.

De los extras opcionales se abordaron dos: tres enfoques alternativos y OpenMP. El clúster en red local se descartó. Exigía las tres máquinas a la vez, con la misma versión de Open MPI y el mismo sistema operativo, y mezclar macOS con Linux lo volvía frágil para el tiempo disponible.

Estructura del informe

Los antecedentes (Parte A) describen DES, la búsqueda secuencial, las rutinas de la base y las primitivas MPI del aviso de llave. El desarrollo (Parte B) presenta las pruebas del naive y los enfoques alternativos con su análisis teórico. Los resultados comparan los programas por llave, explican por qué una sola llave no basta para compararlos y aplican Amdahl y Gustafson. Cierran las conclusiones, las referencias, el catálogo de funciones (Anexo 1) y la bitácora de pruebas (Anexo 2).

Parte A.1 a A.3: DES y la búsqueda secuencial

A.1 ¿Qué es DES?

El *Data Encryption Standard* (DES) es un algoritmo de cifrado simétrico por bloques. La misma llave sirve para cifrar y para descifrar. Se publicó como estándar federal de Estados Unidos en 1977, a partir del cifrado *Lucifer* de IBM, y su última revisión es la FIPS PUB 46-3 de 1999 (NIST, 1999).

Característica	Valor
Tamaño de bloque	64 bits (8 bytes)
Tamaño de llave	64 bits, de los cuales 8 son de paridad: 56 bits efectivos
Espacio de llaves	$2^{56} = 72\,057\,594\,037\,927\,936 \approx 7.2 \cdot 10^{16}$
Estructura	Red de Feistel de 16 rondas
Subllaves	16 subllaves de 48 bits, una por ronda

El último bit de cada uno de los 8 bytes de la llave es de paridad impar y no interviene en el cifrado. Por eso, aunque la llave ocupa 64 bits, la fuerza bruta solo tiene que recorrer 2^{56} combinaciones.

Cifrado de un bloque

El bloque de 64 bits pasa primero por una permutación inicial (IP), que reordena sus bits según una tabla fija. El resultado se parte en dos mitades de 32 bits, L_0 a la izquierda y R_0 a la derecha.

Luego vienen 16 rondas de Feistel. En la ronda i, la nueva mitad izquierda es la derecha anterior, $L_i = R_{i-1}$, y la nueva derecha es $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$. Solo la mitad derecha pasa por la función f; la izquierda solo se combina con un XOR.

Tras la ronda 16 se intercambian las mitades y el bloque queda $R_{16} \,\|\, L_{16}$. Por último, la permutación final IP^{-1}, inversa de la inicial, entrega el bloque cifrado de 64 bits.

La función f

La función f(R, K) recibe los 32 bits de la mitad derecha y los 48 bits de la subllave de la ronda (NIST, 1999; Stallings, 2020). Primero, la expansión E lleva la mitad derecha de 32 a 48 bits duplicando 16 de ellos. Después se combina con la subllave mediante un XOR.

Los 48 bits resultantes se parten en 8 grupos de 6, y cada grupo entra a una caja S. Cada caja convierte 6 bits en 4 con una tabla: los bits de los extremos eligen la fila y los 4 centrales, la columna. Las cajas S son la única parte no lineal de DES y la fuente de su seguridad. Los 32 bits que salen se reordenan con la permutación P.

Generación de subllaves

La permutación PC-1 toma la llave de 64 bits, descarta los 8 bits de paridad y reordena los 56 restantes. Esos 56 bits se dividen en dos mitades de 28, C_0 y D_0.

En cada ronda, C y D rotan a la izquierda una posición en las rondas 1, 2, 9 y 16, y dos posiciones en las demás. En total suman 28, así que al final vuelven a su estado inicial. De los 56 bits de $C_i \parallel D_i$, la permutación PC-2 elige 48, que forman la subllave K_i.

Descifrado

Gracias a la estructura de Feistel, descifrar es el mismo algoritmo con las subllaves en orden inverso, de K_{16} a K_1. La función f no tiene que ser invertible, porque cada ronda se deshace aplicando otra vez el mismo XOR (Stallings, 2020).

Modo ECB y relleno

DES cifra bloques de 8 bytes. Para textos más largos usamos el modo ECB (*Electronic Codebook*), que cifra cada bloque por separado con la misma llave. Nuestro texto, «Esta es una prueba de proyecto 2», mide 32 bytes, es decir, 4 bloques exactos. Si no fuera múltiplo de 8, el último bloque se rellenaría con ceros (función `padded_length`).

ECB no es seguro para mensajes reales. Dos bloques de texto iguales producen dos bloques cifrados iguales, y así se filtran patrones del mensaje. Los modos encadenados como CBC o CTR lo evitan (Stallings, 2020). En este proyecto no importa, porque lo que se ataca es la llave.

DES ante la fuerza bruta

Con 56 bits efectivos, DES quedó al alcance del *hardware* dedicado a finales de los noventa. En 1998, la máquina Deep Crack de la Electronic Frontier Foundation, que costó unos 250 000 dólares, recuperó una llave en 56 horas. En enero de 1999, el DES Challenge III, resuelto por la EFF junto con distributed.net, lo logró en 22 horas y 15 minutos (EFF, 1998). Una sola CPU está muy lejos de eso, como muestra la sección A.3.

A.2 Diagrama de flujo

La figura 1 resume el cifrado de un bloque y la generación de las 16 subllaves. El diagrama se generó con Mermaid desde `docs/img/des-flujo.mmd`.

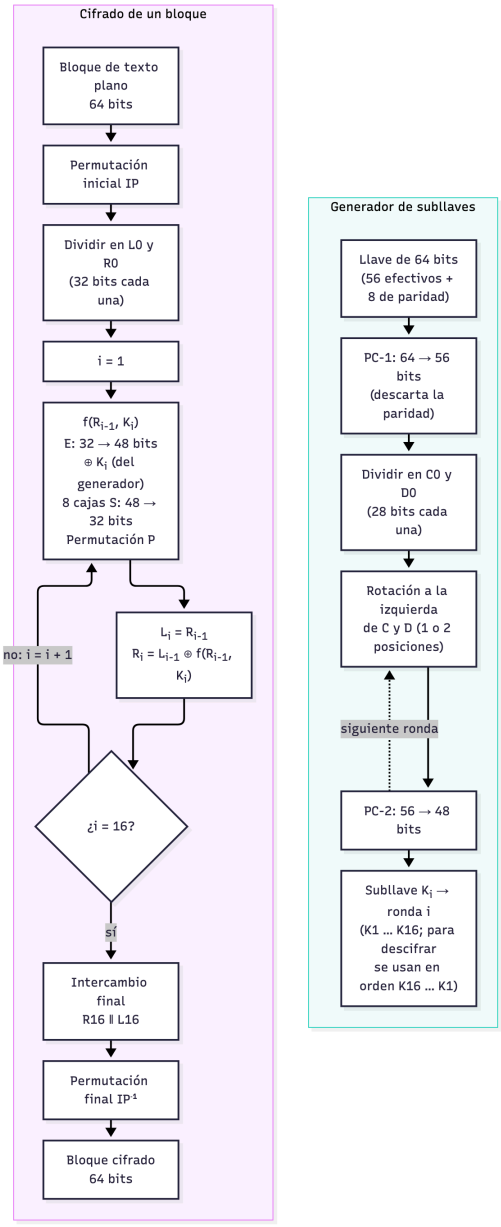


Figura 1. Cifrado de un bloque con DES (izquierda) y generación de las 16 subllaves (derecha).

A.3 Búsqueda secuencial con distintas longitudes de llave

El programa `secuencial` prueba las llaves 0, 1, 2 y así en orden, hasta que el texto descifrado contiene la frase clave. Para medir cómo crece el tiempo con la longitud de la llave se usó el peor caso de cada longitud: con b bits, la llave $2^b - 1$, que obliga a probar las 2^b llaves.

Las corridas las hace el script `medir_longitudes.sh`. En la laptop de Esteban cada longitud se repitió 5 veces y se reporta la mediana. En la Mac de Nico hubo una sola corrida por longitud, que sirve como comparación. Los datos crudos están en la carpeta `resultados/`, en los archivos `secuencial_longitudes_escu.csv` (laptop) y `secuencial_longitudes.csv` (Mac).

Bits	Llaves probadas	Laptop de Esteban (s)	Ritmo (llaves/s)	Mac de Nico (s)
16	65 536	0.044	$1.50 \cdot 10^6$	0.020
18	262 144	0.112	$2.35 \cdot 10^6$	0.071
20	1 048 576	0.370	$2.83 \cdot 10^6$	0.272
22	4 194 304	1.370	$3.06 \cdot 10^6$	1.082
24	16 777 216	5.437	$3.09 \cdot 10^6$	4.322
26	67 108 864	21.939	$3.06 \cdot 10^6$	17.241

El tiempo crece de forma exponencial con la longitud de la llave. Cada bit duplica el espacio, así que cada 2 bits el tiempo se multiplica por 4: de 22 a 24 bits pasa de 1.37 s a 5.44 s, y de 24 a 26 bits, a 21.94 s. Dicho de otro modo, el tiempo es lineal en el número de llaves, porque cada intento cuesta lo mismo.

Con 16 y 18 bits el ritmo es menor. Esas corridas duran milisegundos, y en ese tiempo la CPU todavía no alcanza su frecuencia máxima. Desde 22 bits el ritmo se estabiliza en unos $3.1 \cdot 10^6$ llaves por segundo, y ese es el valor que sirve para extrapolar.

Con ese ritmo, 28 bits tomarían unos 87 s en el peor caso, y la llave media de la campaña, que está en el 62.5 % del espacio, unos 54 s; la campaña midió 56.6 s. Con 56 bits, el peor caso tomaría $2^{56} / 3.1 \cdot 10^6 \approx 2.3 \cdot 10^{10}$ s, más de 700 años. La Mac es alrededor de un 27 % más rápida, pero eso no cambia el orden de magnitud. Por eso las mediciones del proyecto se hacen con 28 bits.

Parte A.4 a A.6: Rutinas de la base y primitivas MPI

A.4 Por qué no compila `rpc/des_crypt.h`

El `bruteforce.c` del curso incluye `<rpc/des_crypt.h>` y usa `ecb_crypt` y `des_setparity`. Esas funciones eran parte de la implementación de Sun RPC que traía la biblioteca de C de GNU (glibc). En glibc 2.32, publicada en 2020, se eliminó por completo Sun RPC junto con `rpcgen` y sus cabeceras (Fedora Project, s. f.; Weimer, 2020). Desde entonces se recomienda la biblioteca externa libtirpc, que no viene instalada por defecto.

El motivo fue que esa implementación no soportaba IPv6, y arreglarlo exigía romper la ABI (Fedora Project, s. f.). Por eso, en una distribución moderna como Ubuntu 22.04, Arch o Fedora, la línea `#include <rpc/des_crypt.h>` falla con «No such file or directory». En macOS tampoco existe.

En lugar de depender de libtirpc, usamos la API DES de OpenSSL (`<openssl/des.h>`, enlazada con `-lcrypto`). Está en cualquier sistema y permite verificar el resultado con `openssl enc -des-ecb` (OpenSSL Project, s. f.). OpenSSL 3 marca esa API como obsoleta porque DES ya no es seguro, pero la mantiene; el aviso se apaga con `-DOPENSSL_SUPPRESS_DEPRECATED`. La tabla siguiente muestra qué rutina de nuestro módulo `des_utils` reemplaza a cada original.

`bruteforce.c` (Sun RPC)	Nuestro proyecto (OpenSSL)
`encrypt(key, ciph, len)`	`encrypt_des(key, buffer, length)`
`decrypt(key, ciph, len)`	`decrypt_des(key, buffer, length)`
`tryKey(key, ciph, len)`	`try_key(key, cipher, length, phrase, workspace)`
`des_setparity(k)`	`DES_set_odd_parity(&k)`
`ecb_crypt(k, buf, len, DES_ENCRYPT)`	`DES_set_key_unchecked` + `DES_ecb_encrypt` por bloque

A.5 Las rutinas de la base

`encrypt_des` y `decrypt_des`

Ambas llaman a la misma función interna, `run_des`, y solo cambian la dirección (`DES_ENCRYPT` o `DES_DECRYPT`). Trabajan sobre el mismo buffer: el resultado sobrescribe la entrada.

Lo primero es armar la llave con `build_des_key`. La llave es un número de 56 bits, pero DES pide 8 bytes en los que el bit menos significativo de cada uno es de paridad. Igual que el `decrypt()` original, los 56 bits se reparten en grupos de 7: el byte i guarda los bits 7i a 7i + 6 en sus posiciones 1 a 7. Después, `DES_set_odd_parity` llena la posición 0 para que cada byte tenga paridad impar.

Con la llave lista, `DES_set_key_unchecked` calcula las 16 subllaves de 48 bits con PC-1, las rotaciones y PC-2. Usamos la versión sin revisión porque la normal rechaza las llaves débiles de DES,

y en fuerza bruta hay que probarlas todas. Por último, `DES_ecb_encrypt` procesa cada bloque de 8 bytes con la misma agenda. La longitud siempre es múltiplo de 8 gracias al relleno con ceros.

`try_key`

Es el corazón de la búsqueda, porque prueba una llave candidata. Copia el cifrado al buffer `workspace` con `memcpy` y le pone un `'\0'` al final para tratarlo como cadena de C. Luego lo descifra con la llave candidata y busca la frase clave con `strstr`.

Si la frase aparece, la llave es la correcta. Con muy poca probabilidad podría ser un falso positivo, y en ese caso el reporte lo marca. El `workspace` lo reserva una sola vez quien llama, en lugar de pedir memoria en cada intento como hacía el original. Así se evitan millones de llamadas a `malloc` y `free`.

`memcpy`

`memcpy(destino, origen, n)` copia `n` bytes de un buffer a otro (ISO/IEC, 2011). Aquí es indispensable porque `decrypt_des` descifra sobre el mismo buffer. Si se descifrara el cifrado original, el primer intento fallido lo destruiría y no quedaría nada que probar con la llave siguiente. Con la copia, el cifrado original queda intacto durante toda la búsqueda.

`strstr`

`strstr(texto, frase)` devuelve un puntero a la primera aparición de `frase` dentro de `texto`, o `NULL` si no aparece (ISO/IEC, 2011). Es la prueba de que la llave es correcta: con una llave errónea, el descifrado es basura y la frase no aparece. Por eso el texto debe terminar en `'\0'`, y por eso `problem_prepare` rechaza de entrada una frase que no esté en el texto original. La figura 2 muestra la secuencia completa de un intento.

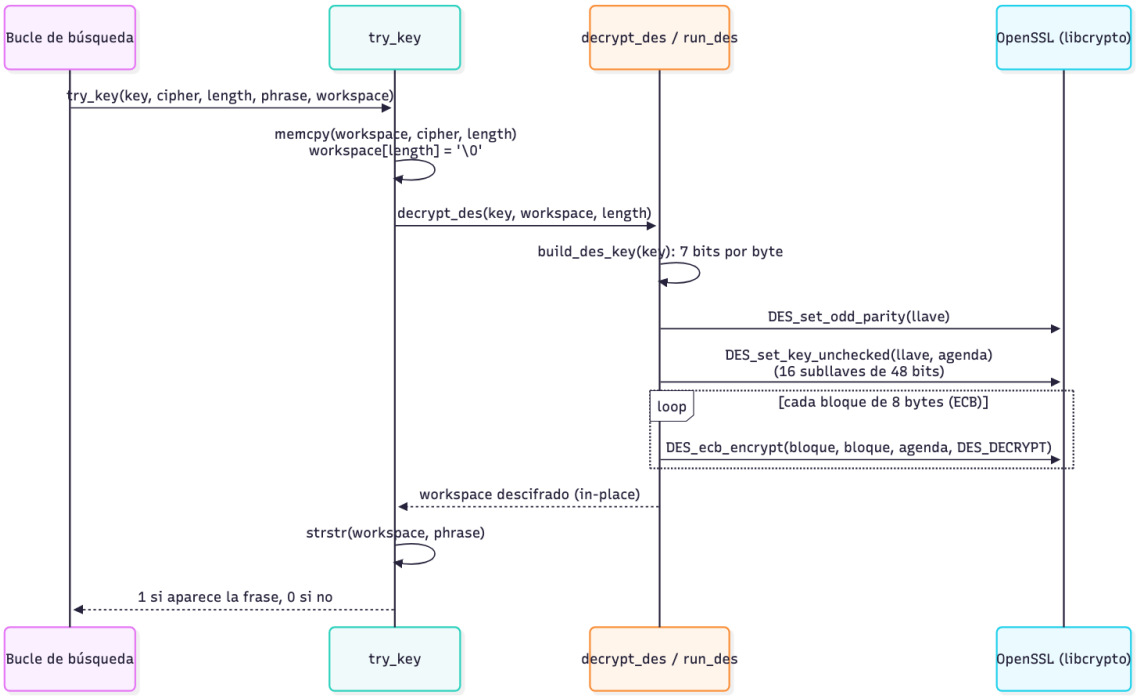


Figura 2. Lo que ocurre en cada llamada a `try_key`.

A.6 Primitivas MPI del aviso de llave

El aviso de que alguien halló la llave usa cuatro primitivas de MPI (MPI Forum, 2023). La tabla resume para qué sirve cada una.

Primitiva	Tipo	Para qué la usamos
`MPI_Irecv`	No bloqueante	Dejar puesta una «antena» que reciba el aviso de llave encontrada
`MPI_Send`	Bloqueante	Que quien halla la llave avise a todos los procesos
`MPI_Wait`	Bloqueante	Cerrar la recepción pendiente antes de terminar
`MPI_Test`	No bloqueante	Revisar, sin detenerse, si ya llegó el aviso

`MPI_Irecv`

`MPI_Irecv(buf, count, datatype, source, tag, comm, &request)` inicia una recepción y regresa de inmediato, sin esperar el mensaje. Devuelve un `MPI_Request` con el que después se consulta o se espera. En el naive, cada proceso la publica antes de empezar a buscar:

```
MPI_Irecv(found_msg, 3, MPI_UINT64_T, MPI_ANY_SOURCE, FOUND_TAG, MPI_COMM_WORLD,
&request);
```

Con `MPI_ANY_SOURCE` el aviso se recibe venga de quien venga, y `found_msg` guarda la llave, el proceso que la halló y sus iteraciones. Mientras tanto, el proceso sigue probando llaves; por eso la

llamamos la antena.

`MPI_Send`

`MPI_Send(buf, count, datatype, dest, tag, comm)` envía un mensaje y no regresa hasta que el buffer se puede reutilizar. El proceso que encuentra la llave envía el aviso a todos, incluido él mismo, para que su propio `MPI_Irecv` también se complete y todos cierren igual. El mensaje mide 24 bytes y cada destino ya tiene su recepción publicada, así que el envío termina casi de inmediato.

`MPI_Wait`

`MPI_Wait(&request, &status)` bloquea hasta que termina la operación no bloqueante asociada. Al salir del bucle, todos los procesos hacen `MPI_Wait` sobre su antena. Así se aseguran de que `found_msg` ya tiene el aviso antes de usarlo y de que no queda nada pendiente al llamar a `MPI_Finalize`. Si un proceso agotó su bloque sin hallar nada, `MPI_Wait` lo deja esperando hasta que otro avise.

Por qué hace falta `MPI_Test`

El `bruteforce.c` original revisa una variable `found` en cada vuelta, pero esa variable solo cambia si la biblioteca MPI hace avanzar la recepción. Open MPI no tiene, por defecto, un hilo que procese mensajes en segundo plano: las comunicaciones solo avanzan dentro de llamadas a MPI. Un proceso que solo ejecuta `try_key` nunca entra a MPI, así que el aviso no se entrega y el proceso termina su bloque completo.

Por eso, cada `POLL_INTERVAL = 4096` llaves, el naive llama a `MPI_Test(&request, &done, MPI_STATUS_IGNORE)`. Esa llamada le da a MPI la oportunidad de avanzar y, si el aviso ya llegó, `done` vale 1 y el proceso sale del bucle. Probar 4096 llaves toma alrededor de 1 ms, así que el costo es despreciable y el retraso para enterarse es de ese orden.

Flujo de mensajes del naive con 4 procesos

En la figura 3, el proceso 1 encuentra la llave y envía el aviso a los 4 procesos. Cada uno lo detecta en su siguiente `MPI_Test` y cierra con `MPI_Wait`. Al final, el proceso 0 reúne las iteraciones con `MPI_Reduce` y reporta el resultado.

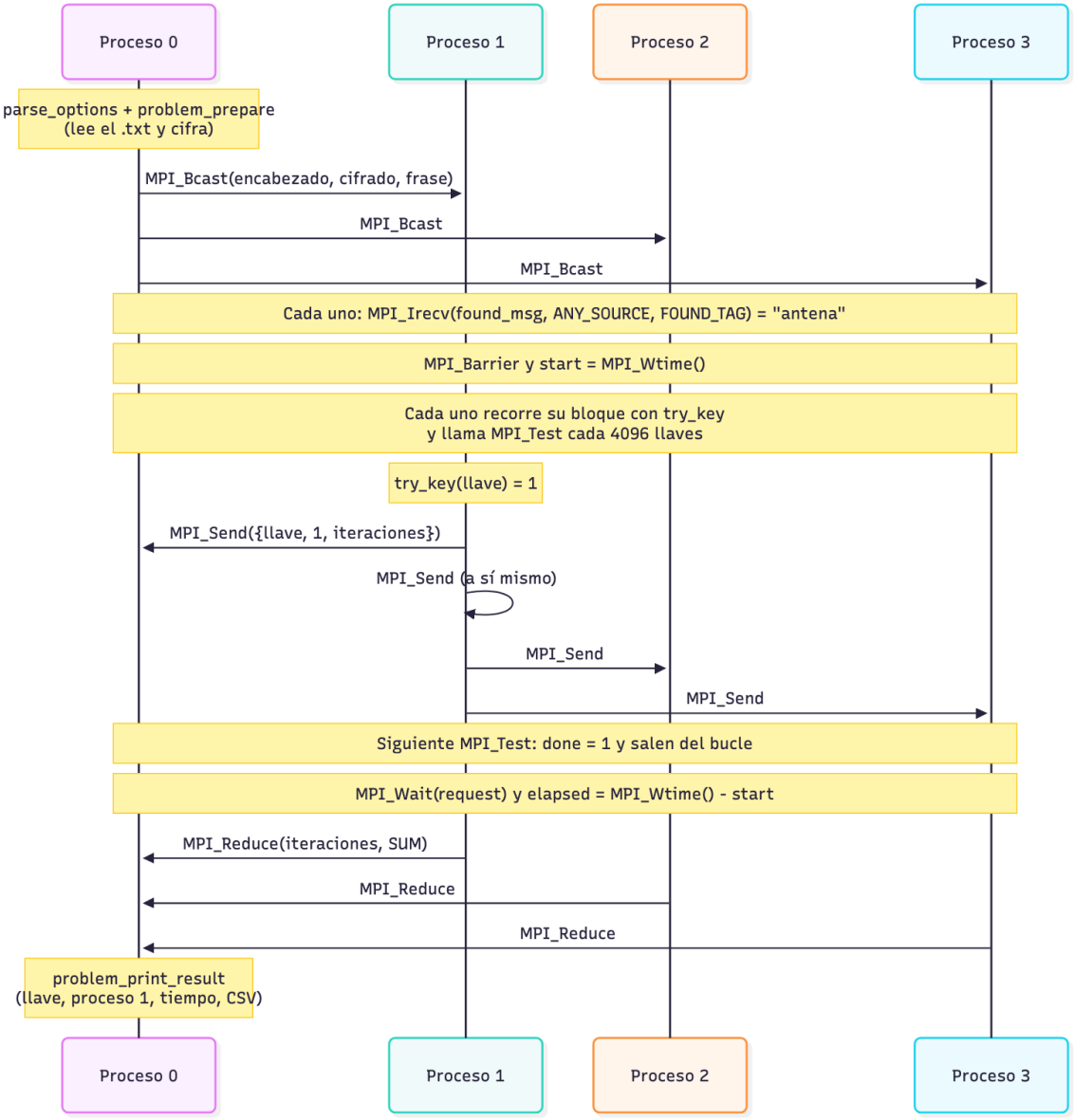


Figura 3. Mensajes del naive con 4 procesos cuando el proceso 1 halla la llave.

Además del aviso, el naive usa `MPI_Bcast` para repartir el problema, porque solo el proceso 0 lee el archivo y cifra. Usa `MPI_Barrier` para que todos arranquen el reloj (`MPI_Wtime`) a la vez y `MPI_Reduce` para sumar las llaves probadas. Cuando hay límite de tiempo (`-t`), el proceso 0 envía el mismo aviso con la llave imposible `UINT64_MAX`, que el reporte traduce a «no encontrada».

Parte B.1 y B.2: Pruebas del cifrado y del naive

Todas las corridas de este capítulo se hicieron en la MacBook Pro de Nico: Apple M4 Pro con 8 núcleos de rendimiento y 4 de eficiencia, macOS 26.6.1, Open MPI 5.0.11 y OpenSSL 3 de Homebrew. Se usó el texto `data/texto.txt` con la frase clave «es una prueba de». Los datos crudos están en `resultados/pruebas_b2.csv` y los genera el script `scripts/pruebas_b2.sh`.

B.1 Cifrar y descifrar desde un `.txt`

El programa `cifrar` carga el texto de `data/texto.txt`, lo cifra con la llave 42 que recibe por parámetro (`-k 42`) y lo descifra de vuelta. El texto descifrado coincide con el original, como se ve en la figura 4.

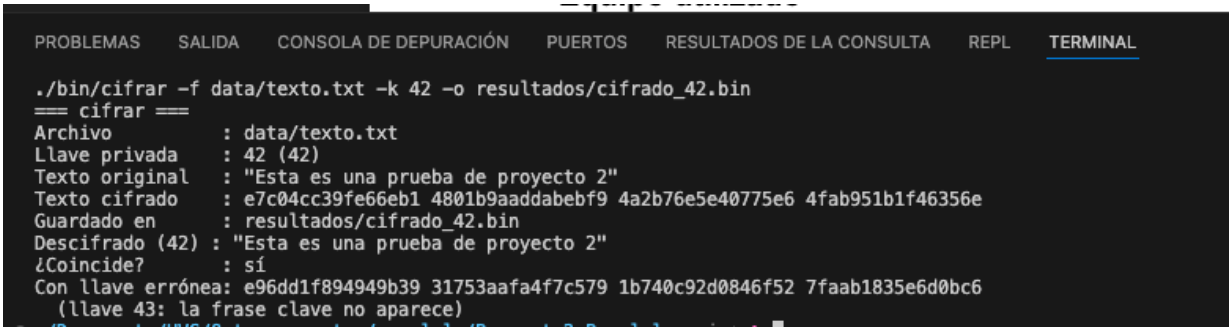


Figura 4. Cifrado y descifrado de `data/texto.txt` con la llave 42.

El cifrado, `e7c04cc39fe66eb1 4801b9aaddabebf9 4a2b76e5e40775e6 4fab951b1f46356e`, es el mismo que produce `openssl enc -des-ecb` con esa llave. Con la llave vecina, la 43, sale basura y la frase clave no aparece. Eso es justo lo que la búsqueda usa para distinguir la llave correcta.

B.2 Naive con 4 procesos contra el secuencial

Las tres llaves del enunciado, con N = 2^{56}, caen en lugares muy distintos del reparto. Con `-np 4`, el naive da a cada proceso un bloque contiguo de N/4 = 2^{54} llaves: el proceso 0 prueba de 0 a N/4 − 1, el proceso 1 de N/4 a N/2 − 1, y así sucesivamente.

Prueba	Llave	Posición
a	123456	Muy al inicio del espacio, dentro del bloque del proceso 0
b	N/4 − 1 = 18014398509481983	Última llave del bloque del proceso 0
c	N/4 = 18014398509481984	Primera llave del bloque del proceso 1

Resultados con 56 bits

Con 56 bits se fijó un límite de 120 s por corrida. La tabla muestra qué programa halló la llave y en cuánto tiempo.

Prueba	Programa	¿La halló?	Proceso	Iteraciones	Llaves probadas	Tiempo
a	secuencial	sí	0	123 457	123 457	0.054 s
a	naive -np 4	sí	0	123 457	492 097	0.034 s
b	secuencial	no (límite)	—	—	459 493 376	120.0 s
b	naive -np 4	no (límite)	—	—	1 689 812 992	120.0 s
c	secuencial	no (límite)	—	—	433 438 720	120.0 s
c	naive -np 4	sí	1	1	12 289	0.0012 s

La prueba a es tan corta que la primera corrida tiene ruido de arranque. Repetida 5 veces, la mediana es 0.032 s para el secuencial y 0.034 s para el naive.

Con 56 bits, las pruebas b y c no terminan en el secuencial, así que su tiempo se estima. En la corrida de 120 s el secuencial probó $r_s \approx 3.83 \cdot 10^6$ llaves/s. Para b, $T_s \approx 2^{54} / r_s \approx 4.70 \cdot 10^9$ s, unos 149 años. Para c el cálculo es prácticamente el mismo.

En el naive, la llave b es la última del bloque del proceso 0, que la alcanza tras 2^{54} intentos. Cada proceso probó unas $3.52 \cdot 10^6$ llaves/s (1 689 812 992 / 4 / 120 s). Entonces $T_p(b) \approx 2^{54} / 3.52 \cdot 10^6 \approx 5.1 \cdot 10^9$ s, unos 162 años.

Resultados con 28 bits

Con `-b 28` los presets escalan ($b = 2^{26} - 1$ y $c = 2^{26}$) y conservan su posición relativa. La llave b sigue siendo la última del proceso 0 y la c, la primera del proceso 1.

Prueba	Programa	Proceso	Iteraciones	Tiempo
b	secuencial	0	67 108 864	18.20 s
b	naive -np 4	0	67 108 864	19.00 s
c	secuencial	0	67 108 865	18.56 s
c	naive -np 4	1	1	0.0012 s

Speedup

El speedup es $S = T_s / T_p$. Además reportamos el speedup de rendimiento, S_{rend}, que divide las llaves por segundo del naive entre las del secuencial. Ese segundo valor mide cuánto trabajo útil hacen los 4 procesos juntos, sin importar dónde cae la llave.

Prueba	T_s	T_p (naive -np 4)	$S = T_s / T_p$	Comentario
a · 56 bits	0.032 s	0.034 s	≈ 0.9	El proceso 0 hace el mismo trabajo que el secuencial
b · 56 bits	≈ 149 años (est.)	≈ 162 años (est.)	≈ 0.9	Ninguna ganancia

Prueba	T_s	T_p (naive -np 4)	$S = T_s / T_p$	Comentario
c · 56 bits	≈ 149 años (est.)	0.0012 s	$\approx 3.9 \cdot 10^{12}$	Aparente: efecto de búsqueda
b · 28 bits	18.20 s	19.00 s	0.96	Ninguna ganancia, medido completo
c · 28 bits	18.56 s	0.0012 s	$\approx 1.6 \cdot 10^{4}$	Aparente: efecto de búsqueda

En la prueba b con 56 bits, los 4 procesos probaron juntos $14.1 \cdot 10^6$ llaves/s, contra $3.83 \cdot 10^6$ del secuencial. Eso da $S_{rend} \approx 3.7$, que no pasa de p = 4. El naive sí trabaja casi cuatro veces más rápido; lo que pasa es que en b todo ese trabajo extra se hace en bloques donde la llave no está.

Reflexión

El tiempo del naive no depende de cuántas llaves hay que probar en total, sino de dónde cae la llave dentro del bloque del proceso que la tiene: $T_p \approx$ (llave − inicio de su bloque + 1) dividido entre el ritmo de un proceso. Las llaves a y b caen en el bloque del proceso 0, que hace los mismos intentos que el secuencial mientras los otros tres trabajan en vano: el speedup ronda 1, o un poco menos por el arranque de MPI y el costo de `MPI_Test`. La llave c es la primera del proceso 1, que la halla en un intento y da un speedup de unos $4 \cdot 10^{12}$.

Las llaves b y c son vecinas, pues difieren en 1, y sin embargo una da el peor resultado posible y la otra uno astronómico. Ese S > p no es superlinealidad real, sino efecto de búsqueda: el naive no hizo el trabajo más rápido, empezó a buscar justo donde estaba la llave, y su speedup de rendimiento no pasa de 3.7. El resultado del naive depende de la suerte y no del algoritmo; los enfoques del capítulo siguiente reparten las llaves para que todos los procesos avancen a la vez por las zonas bajas del espacio, de modo que el speedup sea cercano a p con cualquier llave.

Capturas

Para las capturas, las corridas que no terminan usan un límite de 20 s en lugar de 120 s. Las tablas anteriores salen de la corrida completa de `scripts/pruebas_b2.sh`.

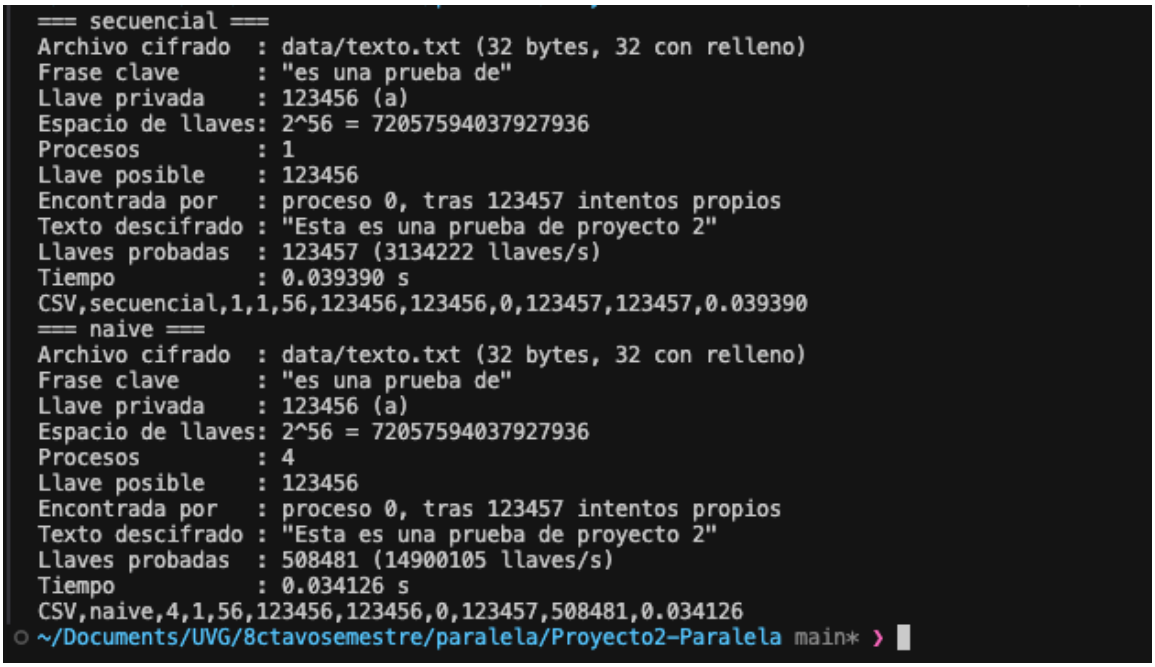


Figura 5. Prueba a: el secuencial y el naive hallan la llave 123456 en el proceso 0 tras 123 457 intentos.

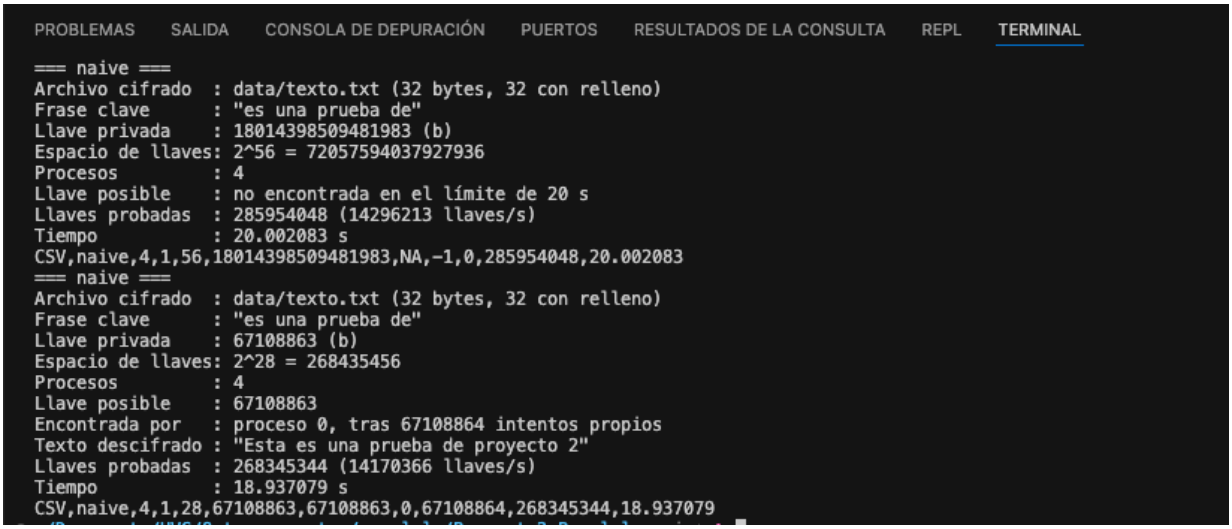


Figura 6. Prueba b: con 56 bits el naive no la halla en el límite; con 28 bits la halla el proceso 0 tras recorrer su bloque completo (2^{26} intentos, 18.9 s).

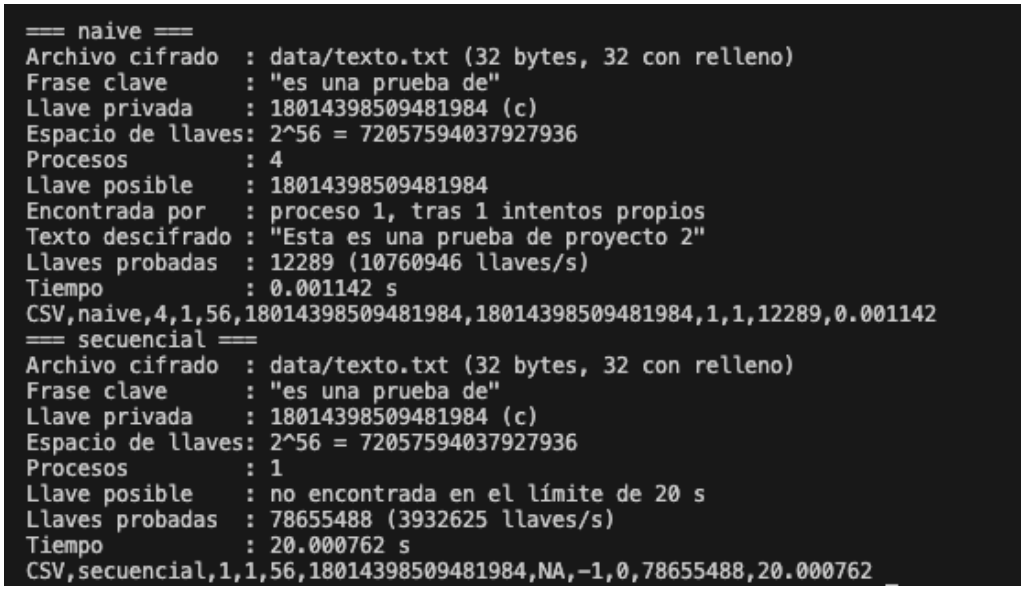


Figura 7. Prueba c: el proceso 1 halla la llave al primer intento (0.001 s), mientras el secuencial no la halla en el límite.

Enfoques de paralelización

Los programas MPI comparten el arranque, el aviso de llave y el reporte en el módulo `search_mpi`; lo único que cambia entre ellos es qué llaves prueba cada proceso y en qué orden. Este capítulo describe cada reparto, predice con una fórmula quién halla la llave y en cuántas iteraciones, y discute qué gana y qué pierde cada uno.

Esqueleto común

Todos los repartos estáticos siguen el mismo esqueleto, que usa las primitivas de la sección A.6. Solo el proceso 0 lee y cifra el texto y lo difunde; cada proceso deja puesta su antena antes de buscar y arranca el reloj después de una barrera.

```
proceso 0: lee parámetros y archivo, cifra el texto
MPI_Bcast(encabezado, cifrado, frase)           # errores: salen una vez y todos cierran
MPI_Irecv(aviso)                                # la "antena", puesta antes de buscar
MPI_Barrier; inicio = MPI_Wtime()
para cada llave k del reparto de este proceso:
    cada POLL_INTERVAL = 4096 llaves: MPI_Test(antena); si llegó el aviso, salir
    si try_key(k): MPI_Send(aviso {k, rango, iteraciones}) a todos; salir
MPI_Wait(antena); T_p = MPI_Wtime() - inicio
MPI_Reduce(llaves probadas, SUMA) y el proceso 0 reporta
```

En lo que sigue, $N = 2^{bits}$ es el número de llaves, p el de procesos y k la llave buscada. Las iteraciones son las llaves que probó el proceso que halló la llave. El secuencial prueba k + 1 llaves, así que cada reparto predice, sin necesidad de medir, un speedup por iteraciones S_{iter} = (k + 1) / (iteraciones del que la halla).

La figura 8 compara qué llaves prueba cada proceso en los tres repartos estáticos, con un espacio pequeño de 16 llaves.

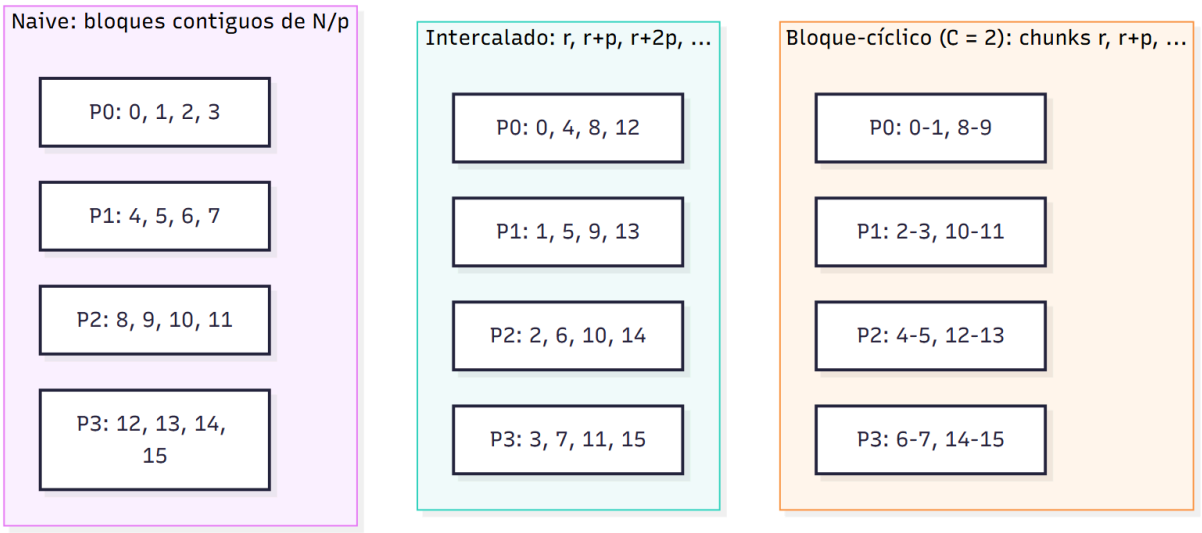


Figura 8. Llaves que prueba cada proceso en los repartos estáticos, con 16 llaves, p = 4 y chunks de C = 2.

Naive: bloques contiguos

El naive es el reparto del `bruteforce.c` del curso y sirve de referencia. El espacio se parte en p bloques de N/p llaves, y el proceso r recorre el suyo de principio a fin.

```
bloque = N / p
inicio = r · bloque;  fin = (r == p - 1) ? N : inicio + bloque
para k = inicio .. fin - 1: probar k
```

La llave la halla el proceso r = ⌊k / (N/p)⌋, tras k − r·N/p + 1 iteraciones. Por eso S_{iter} = (k + 1) / (k − r·N/p + 1). Ese valor va de 1, cuando la llave cae en el bloque del proceso 0, a cerca de N/p, cuando la llave es la primera de un bloque que no es el primero.

Su ventaja es la simpleza: recorre llaves contiguas y no necesita más comunicación que el aviso. Su desventaja es que el speedup depende por completo de dónde cae la llave: los p − 1 procesos que no la tienen trabajan en vano, y con llaves pequeñas, que son las que más probablemente elige una persona, no gana nada.

Intercalado

En el intercalado, el proceso r prueba las llaves r, r + p, r + 2p, y así hasta el final. Todos los procesos avanzan a la vez por las llaves bajas y por las altas.

```
para k = r, r + p, r + 2p, ... < N: probar k
```

La llave la halla el proceso k mod p, tras ⌊k/p⌋ + 1 iteraciones. Entonces S_{iter} = (k + 1) / (⌊k/p⌋ + 1), que vale casi exactamente p y nunca lo supera.

Su gran ventaja es que el speedup ronda p para cualquier llave, sin comunicación extra y cambiando solo dos líneas respecto al naive. Como desventaja, cada proceso salta de p en p y no aprovecha llaves contiguas, aunque aquí eso no importa porque cada llave genera su propia agenda de subllaves. Además es un reparto estático: si un núcleo es más lento, todos lo esperan.

Bloque-cíclico

El bloque-cíclico parte el espacio en chunks de $C = 2^c$ llaves, con `-c` igual a 16 por defecto. El proceso r recorre los chunks r, r + p, r + 2p, y dentro de cada uno avanza en orden. Es el punto medio entre los dos anteriores: con C = N/p es el naive y con C = 1 es el intercalado.

```
para j = r, r + p, r + 2p, ... < N / C:
    para k = j·C .. (j + 1)·C - 1: probar k
```

Si $j = \lfloor k/C \rfloor$ es el chunk de la llave, la halla el proceso j mod p tras $\lfloor j/p \rfloor \cdot C + (k \bmod C) + 1$ iteraciones. El speedup se acerca a p cuando k es mucho mayor que p·C. Con llaves diminutas se parece al naive: la llave 123456, con $C = 2^{16}$, cae en el chunk 1, del proceso 1, en la posición 57 920, y da $S_{iter} \approx 2.13$.

Este reparto combina el equilibrio del intercalado con el recorrido contiguo del naive, y el tamaño de chunk permite moverse entre ambos. A cambio, agrega un parámetro, y con chunks grandes respecto a la llave se pierde la consistencia.

Maestro-trabajador

En este enfoque, el proceso 0 es el maestro y no busca: reparte chunks de C llaves a quien los pide. Los procesos 1 a p − 1 son trabajadores, que piden un chunk, lo prueban y piden otro. Es el único reparto dinámico, porque un núcleo más lento simplemente pide menos chunks.

```
maestro:                                  trabajador:
  activos = p - 1                           pedir; recibir respuesta
  mientras activos > 0:                     mientras la respuesta sea WORK:
    recibir pedido (MPI_ANY_SOURCE)           pedir el siguiente (MPI_Send + MPI_Irecv)
    si es mensaje final (FOUND o DONE):       probar el chunk actual
      si FOUND: T_p = ahora; parar = 1          si halló k: enviar FOUND {k, iter}
      activos -= 1                                        MPI_Wait(respuesta) y terminar
    si no: responder STOP si parar o no       MPI_Wait(respuesta adelantada)
           quedan chunks; si no, WORK         enviar DONE
```

Dos invariantes evitan cuelgues. Cada pedido recibe exactamente una respuesta, y cada trabajador termina con un único mensaje final, FOUND o DONE, que no espera respuesta. Como MPI no adelanta mensajes del mismo origen con la misma etiqueta, cuando el maestro lee el mensaje final de un trabajador ya atendió todos sus pedidos anteriores.

El trabajador pide el chunk siguiente antes de probar el actual. Ese doble buffer, con `MPI_Send`, `MPI_Irecv` y `MPI_Wait`, esconde la ida y vuelta con el maestro detrás del cálculo. La figura 9 muestra

el intercambio de mensajes con tres procesos.

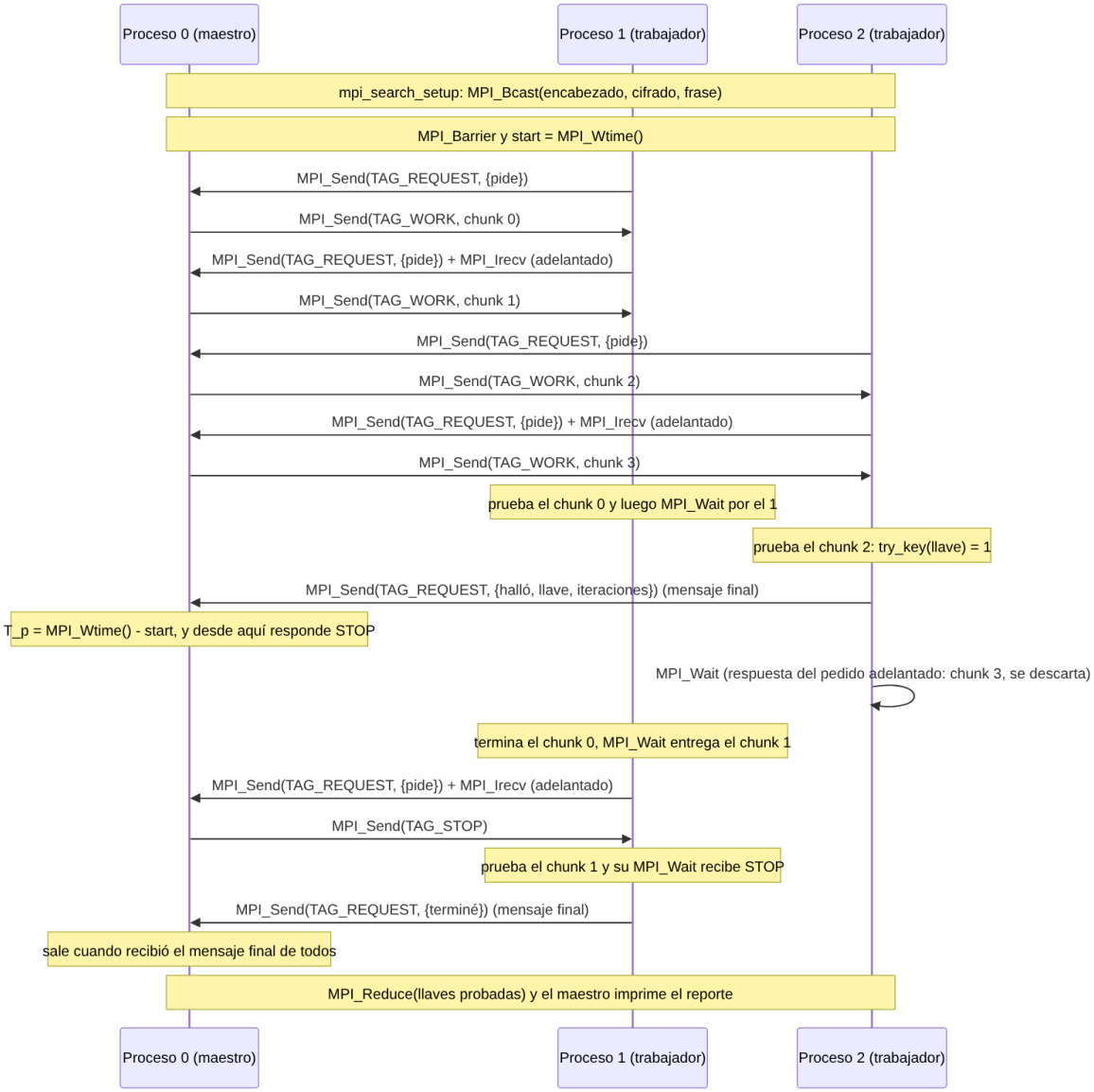


Figura 9. Mensajes del maestro-trabajador con un maestro y dos trabajadores.

Con p − 1 trabajadores que van al mismo ritmo, los chunks se reparten casi en round-robin entre ellos. Por eso se comporta como el bloque-cíclico con p − 1 procesos: $S_{iter} \approx p - 1$, unos 3 con `-np 4`, y su eficiencia ideal es (p − 1)/p, 0.75 con `-np 4`.

Su ventaja es el balanceo automático cuando los núcleos son desiguales o la máquina está compartida, y respeta el límite `-t` con una resolución de un chunk. Su desventaja es que el maestro no busca, así que con pocos procesos se pierde 1/p del poder de cómputo: con `-np 2` hay un solo trabajador y no puede ganarle al secuencial. Con más procesos, (p − 1)/p se acerca a 1.

Híbrido MPI + OpenMP

El híbrido es el intercalado con dos niveles: p procesos MPI y T hilos de OpenMP por proceso, según `OMP_NUM_THREADS`. El hilo t del proceso r es el trabajador global g = r·T + t, y prueba g, g + pT, g + 2pT, y así sucesivamente. Es el mismo intercalado, ahora con w = p·T trabajadores, como muestra la figura 10.

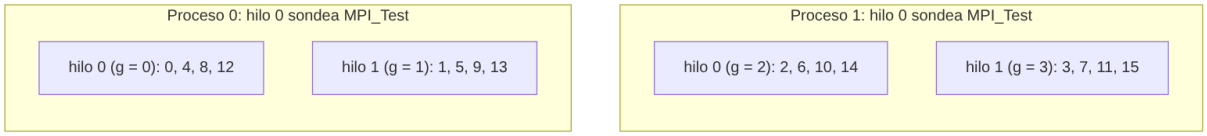


Figura 10. Llaves que prueba cada hilo del híbrido con 16 llaves, p = 2 y T = 2.

```
MPI_Init_thread(MPI_THREAD_FUNNELED)            # solo el hilo principal llama a MPI
mpi_search_setup; T = omp_get_max_threads() del proceso 0, MPI_Bcast(T)
antena: MPI_Irecv; MPI_Barrier; inicio = MPI_Wtime()
#pragma omp parallel num_threads(T) reduction(+: llaves)
    g = r·T + t;  workspace propio
    para k = g, g + pT, g + 2pT, ... < N:
        cada 4096 llaves:
            si t == 0 y MPI_Test(antena): stop = 1     # aviso de otro proceso
            si stop: salir
        si try_key(k): critical { guardar k, iteraciones }; stop = 1; salir
si un hilo la halló: MPI_Send(aviso) a todos        # fuera de la región paralela
MPI_Wait(antena); MPI_Reduce(llaves); reporte con hilos = T
```

La función `try_key` es reentrante, porque la agenda de subllaves de DES es local, así que basta un `workspace` por hilo. La bandera de parada es un `atomic_int` de C11, y los hilos la leen cada 4096 llaves, igual que el sondeo MPI.

MPI se inicia con `MPI_THREAD_FUNNELED`. Solo el hilo 0, que es el principal, llama a `MPI_Test`, y el aviso sale después de la región paralela. Así no hace falta que el MPI instalado acepte llamadas desde varios hilos. El número de hilos lo decide el proceso 0 y lo difunde, para que todos usen el mismo paso p·T.

Al medir hay un detalle importante. Open MPI 5 ata cada proceso a un solo núcleo por defecto, y entonces los T hilos de un proceso se pelean ese núcleo y el híbrido no escala. En el i9, con 4 procesos, 1, 2 y 4 hilos tardaban casi lo mismo. Por eso el híbrido se lanza con `mpirun --bind-to none`, como lo hacen `verificar.sh` y `campana.sh` (Open MPI Project, s. f.). En macOS Open MPI no ata procesos, así que allá no cambia nada.

La llave la halla el hilo g = k mod pT, tras $\lfloor k/(pT) \rfloor + 1$ iteraciones. Entonces $S_{iter} = (k + 1) / (\lfloor k/(pT) \rfloor + 1)$, que ronda p·T y nunca lo supera. Con T = 1 es exactamente el intercalado, y para la eficiencia, el costo y la efectividad se usa w = p·T.

El híbrido usa todos los núcleos con pocos procesos MPI, lo que significa menos mensajes y menos memoria que `intercalado -np p·T`, y conserva un speedup consistente para cualquier llave. A

cambio, agrega un nivel de sincronización y depende de cómo el lanzador ata los procesos. Si hay más trabajadores que núcleos de rendimiento, los hilos que caen en núcleos de eficiencia frenan al resto, porque el reparto es estático. Su consistencia con la versión sin OpenMP se comprueba en el capítulo de resultados.

Resumen teórico

La tabla reúne, para cada programa, quién tiene la llave k y tras cuántos intentos propios la halla. Con $N = 2^b$ llaves, p procesos, T hilos por proceso y chunks de $C = 2^c$ llaves:

Programa	Quién tiene la llave	Intentos para hallarla	S_{iter} con `-np 4`
Secuencial	el único proceso	$k + 1$	1
Naive	$r = \lfloor k / (N/p) \rfloor$ (el último absorbe el residuo)	$k - r \cdot (N/p) + 1$	de 1 a cerca de N/4
Intercalado	$r = k \bmod p$	$\lfloor k / p \rfloor + 1$	≈ 4
Bloque-cíclico	$r = j \bmod p$, con $j = \lfloor k / C \rfloor$	$\lfloor j / p \rfloor \cdot C + (k \bmod C) + 1$	≈ 4 si $k \gg p \cdot C$
Maestro-trabajador	el trabajador al que le tocó el chunk j	$\approx \lfloor j / (p - 1) \rfloor \cdot C + (k \bmod C) + 1$	≈ 3
Híbrido	hilo $g = k \bmod (p \cdot T)$	$\lfloor k / (p \cdot T) \rfloor + 1$	$\approx 4 \cdot T$

Con las llaves del enunciado (56 bits, `-np 4`, $C = 2^{16}$), esas fórmulas dan los S_{iter} siguientes:

Llave	Naive	Intercalado	Bloque-cíclico	Maestro-trabajador
$a \cdot 123456$	1	4.00	2.13	2.13
$b \cdot N/4 - 1$	1	4	4	3
$c \cdot N/4$	$1.8 \cdot 10^{16}$ (aparente)	4	4	3
fácil $\cdot N/2 + 1$	$1.8 \cdot 10^{16}$ (aparente)	4	4	3
media $\cdot N/2 + N/8$	5 (aparente)	4	4	3
difícil $\cdot \lceil N/7 + N/13 \rceil$	1	4	4	3

En el naive, la difícil (cerca de $0.22 \cdot N$) y la b caen en el bloque del proceso 0, que hace las mismas $k + 1$ iteraciones que el secuencial. La c es la primera llave del proceso 1 y la fácil la segunda del proceso 2, así que las hallan casi al instante. La media cae a la mitad del bloque del proceso 2, que recorre $N/8 + 1$ llaves frente a las $5N/8 + 1$ del secuencial.

Los presets escalan con N y cada llave conserva su posición relativa, así que con los 28 bits de las mediciones la tabla es la misma, salvo c y fácil en el naive, que valen cerca de $6.7 \cdot 10^7$. Por eso medir con menos bits es representativo.

Si la llave fuera uniforme al azar entre 0 y N, el naive y el intercalado tendrían el mismo tiempo esperado: en ambos, el proceso que la tiene recorre en promedio la mitad de N/p llaves, de modo que $E[T_p] = N/(2p)$ frente a $E[T_s] = N/2$. Lo que gana un reparto alternativo no es el promedio, sino que el

speedup deje de depender de dónde cae la llave: cerca de p, o de p − 1, para cualquier llave, en lugar de algo entre 1 y 10^{16}.

Análisis de speedup y del comportamiento según la llave

Los números de este capítulo salen de la campaña en la laptop de Esteban (i9-13980HX), la máquina oficial del informe, con 28 bits, 5 repeticiones por caso y la mediana de cada caso. El detalle está en el Anexo 2 y en `resultados/analisis_escu.md`.

Las fórmulas son las del curso (Barlas, 2015; Pacheco, 2011). El speedup es $S = T_s / T_p$, la eficiencia $E = S / w$, el costo $C = w \cdot T_p$ y la efectividad $F = S / C$, con $w = p \cdot h$ trabajadores. T_s es siempre el programa `secuencial` con la misma llave y los mismos bits. Además se usan dos speedups auxiliares: el de iteraciones, $S_{iter} = (k + 1) / (\text{iteraciones del que la halla})$, que solo depende del reparto, y el de rendimiento, S_{rend}, que compara llaves probadas por segundo y solo depende del hardware.

Lo teórico frente a lo medido

La tabla pone, para cada llave, el S_{iter} que predice el resumen teórico del capítulo de enfoques junto al speedup medido con `-np 4`.

Llave (28 bits)		Naive	Intercalado	Bloque-cíclico	Maestro-trabajador
fácil · k = 134 217 729	S_{iter} (teoría)	$6.7 \cdot 10^7$	4	4	3
	S medido	11 087 (aparente)	3.19	3.21	2.30
media · k = 167 772 160	S_{iter} (teoría)	5	4	4	3
	S medido	3.95	3.25	3.21	2.32
difícil · k = 58 996 804	S_{iter} (teoría)	1	4	4	3
	S medido	0.79	3.28	3.27	2.35

En todas las corridas, las iteraciones del proceso que halló la llave coinciden exactamente con las fórmulas: el reparto prueba las llaves que debe. La diferencia entre S_{iter} y el S medido es, entonces, pura pérdida de velocidad por llave, que se analiza en las fugas de eficiencia. La figura 11 muestra los speedups medidos.

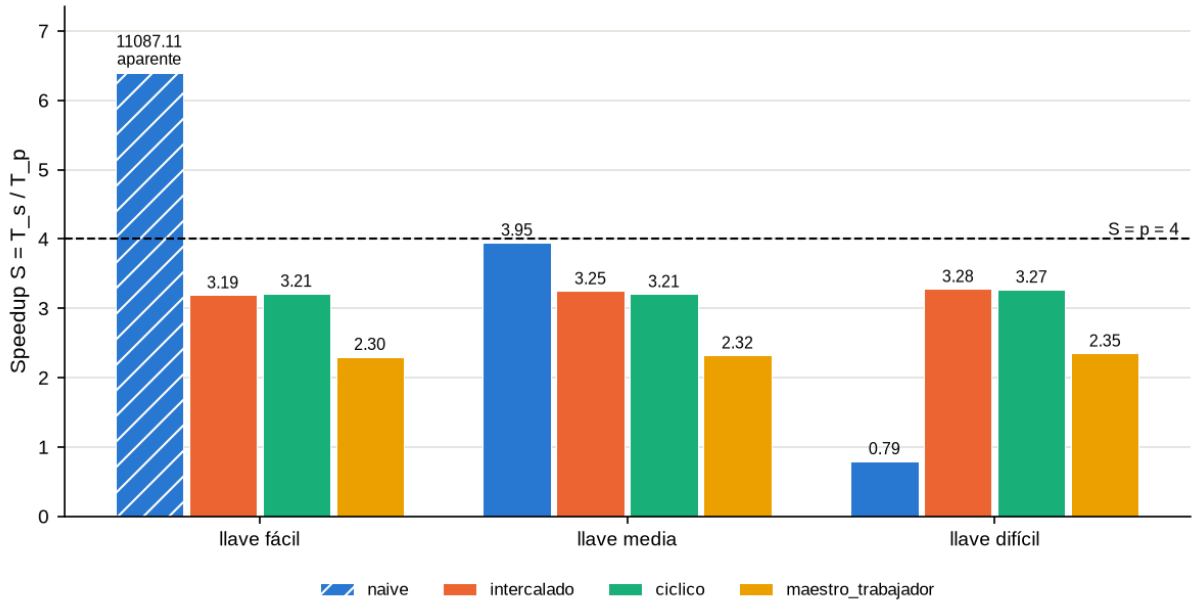


Figura 11. Speedup de cada programa por llave con 4 procesos. La barra del naive con la llave fácil está recortada.

El naive depende de la llave y su superlineal es aparente

Con los mismos 4 procesos, el speedup del naive va de 0.79 a 11 000. La llave fácil es la segunda del bloque del proceso 2, que la halla en 2 intentos y 4 ms, casi todo arranque; con 56 bits el speedup sería del orden de 10^{16}. La media cae a la mitad del bloque del proceso 2: la teoría da 5 y se midieron 3.95, porque cada uno de los 4 procesos activos va al 79 % del ritmo del secuencial (5 × 0.79 ≈ 3.95). La difícil cae en el bloque del proceso 0, que hace las mismas iteraciones que el secuencial: S = 0.79, cuatro procesos tardan más que uno.

En cambio, el intercalado da de 3.19 a 3.28 en las tres llaves y el bloque-cíclico de 3.21 a 3.27, una variación menor al 3 %. El maestro-trabajador da de 2.30 a 2.35, también parejo; con 3 trabajadores su techo es 3, y respecto a ese techo su eficiencia es 2.32 / 3 = 0.77, igual a la de los otros.

El S > p del naive no es superlinealidad real, por tres razones. Primero, el naive no prueba llaves más rápido: S_{rend} nunca supera w en toda la campaña (máximo 3.37 con w = 4 y 10.05 con w = 16), y en el naive con la llave media vale 3.16; el proceso 2 solo tuvo que recorrer 1/8 del espacio. Segundo, es el efecto de búsqueda del ejercicio 9 de la clase; las otras dos causas no aplican, porque el cifrado de 32 bytes cabe en caché en ambos casos y no hay sobrecarga que se reduzca al partir el problema. Tercero, en promedio el efecto desaparece: como se vio en el resumen teórico, $E[T_p] = N/(2p)$ en el naive y en el intercalado. Ningún enfoque alternativo superó S = w, y el análisis automático, que marca cualquier S o S_{rend} mayor que w, no levantó alertas.

La llave 60 000 000: por qué fue un mal ejemplo para comparar

En la comparación que presentamos en clase usamos una sola llave, k = 60 000 000 con 26 bits y `-np 4`, y el naive tardó menos que el intercalado. No es un error del código: es la llave la que decide quién gana. Con 4 núcleos ocupados, cada proceso prueba cerca de 2.5 millones de llaves por segundo en todos los programas, así que gana el que necesita menos intentos según las fórmulas del resumen teórico.

Con 26 bits, N = 67 108 864 y cada bloque del naive mide N/4 = 16 777 216. La llave cae en el último cuarto del espacio.

Programa	Proceso que la halla	Intentos	Tiempo medido	Speedup
Secuencial	0	60 000 001	18.97 s	1.00
Naive	3	9 668 353	3.71 s	5.11
Intercalado	0	15 000 001	5.87 s	3.23

El proceso 3 del naive empieza en 3N/4 = 50 331 648 y solo recorre 9 668 353 llaves. En el intercalado, la llave le toca al proceso 0 (60 000 000 mod 4 = 0), que antes tiene que probar sus 15 000 000 llaves menores. La razón de intentos es 15 000 001 / 9 668 353 = 1.55 y la de tiempos 5.87 / 3.71 = 1.58: la diferencia sale entera del número de intentos, no de que el intercalado pruebe llaves más despacio. El speedup de 5.11 con 4 procesos es la señal de alarma: el proceso 3 se saltó, por su posición, las 50 millones de llaves que el secuencial sí probó.

Para cualquier llave del último cuarto, k = 3N/4 + r con 0 ≤ r < N/4, el naive necesita r + 1 intentos y el intercalado 3N/16 + r/4 + 1. El naive gana si r < 3N/16 + r/4, es decir, si r < N/4, lo que siempre se cumple: en todo el último cuarto el naive le gana al intercalado.

La figura 12 dibuja las fórmulas para todas las llaves. El naive es un serrucho, porque cada proceso empieza su bloque desde cero: la llave cuesta poco al inicio de un bloque y mucho al final. El intercalado es una recta k/4 que sube pareja. Las rectas se cruzan y, como muestra el fondo, cada uno gana en la mitad del espacio. Las líneas verticales marcan la llave de la clase, donde el naive queda abajo, y una llave del primer cuarto, donde el naive coincide con el secuencial y el intercalado necesita la cuarta parte.

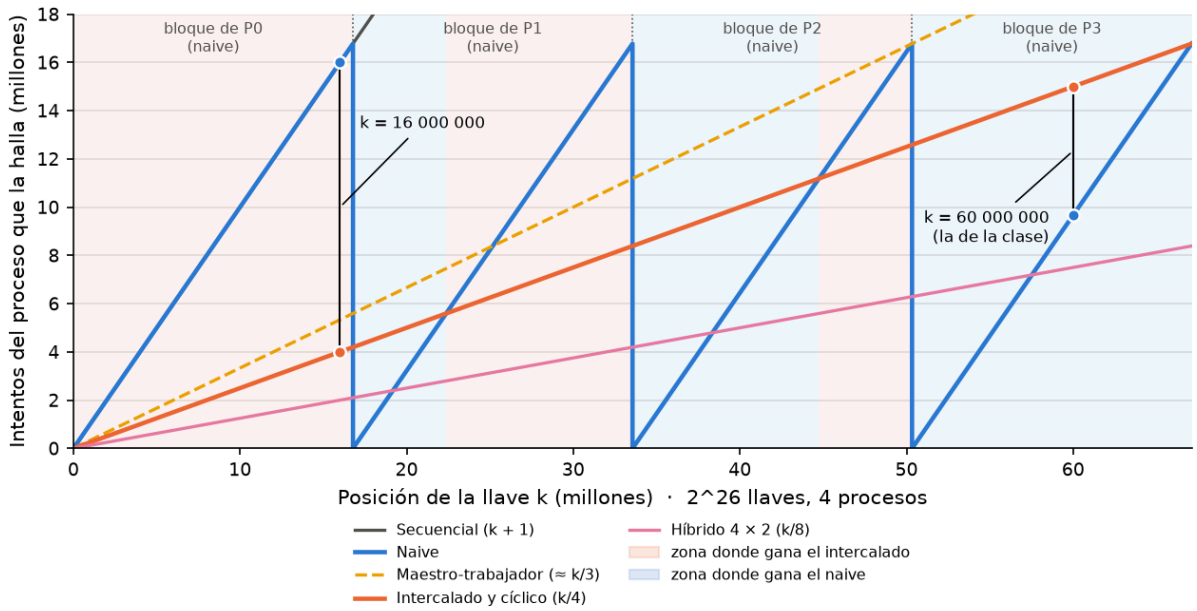


Figura 12. Intentos que necesita el proceso que halla la llave, según dónde cae, con 2^{26} llaves y 4 procesos. El fondo azul marca dónde gana el naive y el naranja dónde gana el intercalado.

Con qué llaves había que comparar

Una sola llave no compara algoritmos, compara posiciones. Lo justo es correr todos los programas con las mismas llaves, pero con varias que cubran el espacio, y comparar el promedio y cuánto varía. Con 26 bits y `-np 4`, la mitad de cada cuarto basta para verlo:

Llave	Dónde cae	Intentos del naive	Intentos del intercalado	Gana
8 388 608	mitad del 1.er cuarto	8 388 609	2 097 153	intercalado (4 veces)
25 165 824	mitad del 2.º cuarto	8 388 609	6 291 457	intercalado
41 943 040	mitad del 3.er cuarto	8 388 609	10 485 761	naive
58 720 256	mitad del 4.º cuarto	8 388 609	14 680 065	naive
Promedio		8 388 609	8 388 609	empate

Cada uno gana en la mitad de los casos y en promedio empatan; sobre todas las llaves posibles, el naive gana en exactamente el 50 %. Los presets b y c del enunciado son los extremos: con b = N/4 − 1, el proceso 0 del naive recorre todo su bloque, igual que el secuencial, contra la cuarta parte en el intercalado; con c = N/4, el proceso 1 del naive la halla en un intento. Un contraejemplo medido, k = 16 000 000: el intercalado tardó 1.70 s y el naive 6.86 s, más que el propio secuencial.

Prueba con 16 llaves al azar

Para no depender de una posición, se corrieron todos los programas con las mismas 16 llaves al azar de 26 bits (`resultados/campana_escu_20261014-2308.csv`). El rango de speedup excluye las dos llaves menores de 2 millones, donde el arranque pesa más que la búsqueda.

Programa	Promedio	Mínimo – máximo	Speedup medio	Speedup mínimo – máximo
Secuencial	11.21 s	0.11 – 22.98 s	1.00	—
Maestro-trabajador	4.62 s	0.03 – 9.38 s	2.58	2.27 – 2.51
Naive	4.34 s	0.13 – 6.40 s	2.87	0.83 – 7.83
Intercalado	3.33 s	0.03 – 6.76 s	3.42	3.00 – 3.53
Bloque-cíclico	3.32 s	0.03 – 6.78 s	3.44	—
Híbrido, 1 hilo	3.28 s	0.03 – 6.69 s	3.48	—
Híbrido, 2 hilos	1.83 s	0.01 – 3.71 s	6.29	—

El naive le ganó al intercalado en 7 de las 16 llaves, todas del tercer o del último cuarto, y fue más lento que el secuencial en otras 7, todas del primer cuarto. Su promedio salió algo peor que el del intercalado porque la muestra trajo 7 llaves del primer cuarto en vez de las 4 esperadas; con llaves uniformes, ambos promedian lo mismo.

La figura 13 muestra esas corridas. Los puntos del naive caen donde dice la figura 12: cerca de 1 en el bloque de P0, alrededor de 1.7 en el de P1 y entre 3.4 y 7.8 en los dos últimos, varias veces por encima del máximo de 4. Los del intercalado y el bloque-cíclico forman una franja pareja cerca de 3.4, los del maestro-trabajador otra cerca de 2.4 y los del híbrido, con el doble de trabajadores, otra cerca de 6.2. Las estrellas son la llave 60 000 000 de la clase.

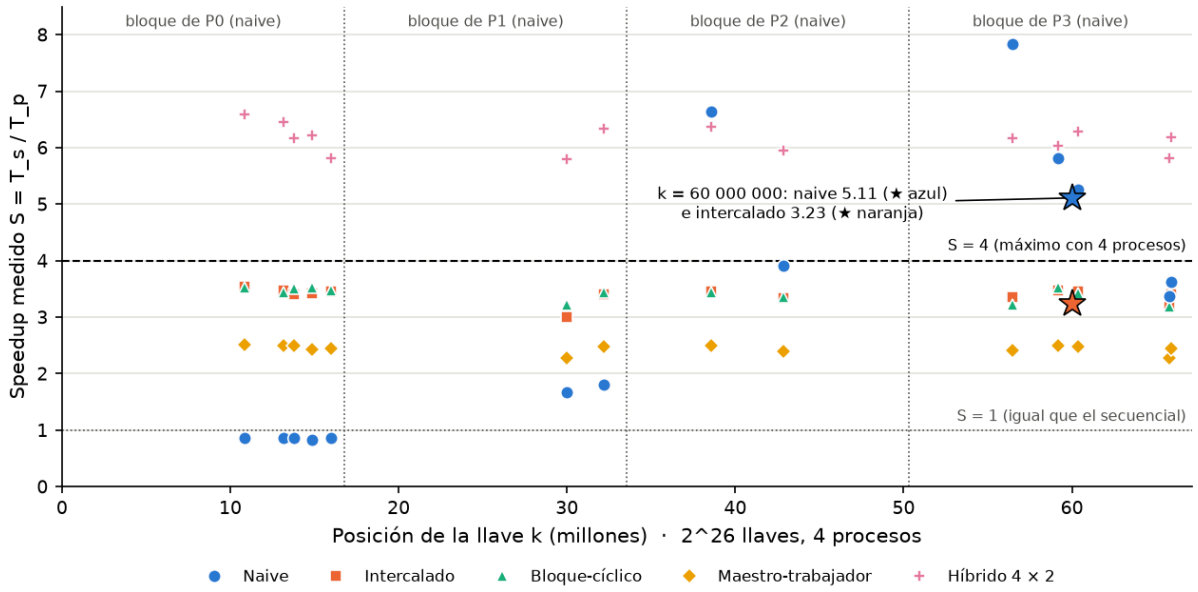


Figura 13. Speedup medido de cada programa con las 16 llaves al azar de 26 bits y 4 procesos (el híbrido con 2 hilos por proceso), según dónde cae la llave.

El intercalado no es mejor porque gane con cada llave, sino porque su speedup no depende de la suerte: queda entre 3.0 y 3.5 con cualquier llave, mientras el del naive va de 0.83 a 7.83. Ningún reparto estático le gana al naive con todas las llaves, porque todos prueban p llaves por paso y el promedio N/(2p) ya es el mínimo posible; para mejorarlo siempre hay que probar más llaves por

segundo, como el híbrido con 8 trabajadores. El maestro-trabajador queda segundo peor porque con `-np 4` solo buscan 3 procesos.

Fugas de eficiencia

Con `-np 4`, el intercalado tiene E ≈ 0.81 y S_{rend} ≈ S (3.26 frente a 3.25): no se desperdician llaves, sino que cada proceso prueba llaves más despacio que el secuencial solo. La tabla ubica cada fuga en nuestro problema.

Fuga	En nuestro problema
Parte secuencial (η_f)	Leer, cifrar y difundir el texto toma milisegundos. En el maestro-trabajador pesa el maestro entero, 1/p del cómputo
Comunicación (η_c)	Un `MPI_Test` cada 4096 llaves y un aviso al final: despreciable. En el maestro-trabajador, un pedido por chunk, oculto en parte por el pedido adelantado
Desbalance (η_b)	Nulo en los repartos estáticos, porque todos reciben N/p llaves; en el bloque-cíclico, como mucho un chunk. En el naive, respecto a la llave: p − 1 procesos trabajan donde no está
Redundancia (η_r)	Ninguna: cada llave se prueba una sola vez
Hardware	La que domina. Con un núcleo activo el i9 sube al turbo máximo; con 4 u 8 baja la frecuencia, y pasados 8 trabajadores se comparten núcleos por *hyperthreading* o entran los de eficiencia

Barrido de procesos, Amdahl y Gustafson

La tabla resume el barrido con la llave media. La figura 14 lo compara con la recta ideal y con la curva de Amdahl.

Programa	p	T_p (s)	S	E	C (s)	F
secuencial	1	56.57	1.00	1.00	56.57	0.018
intercalado	2	32.57	1.74	0.87	65.13	0.027
intercalado	4	17.41	3.25	0.81	69.65	0.047
intercalado	8	11.30	5.01	0.63	90.40	0.055
maestro-trabajador	2	67.13	0.84	0.42	134.27	0.006
maestro-trabajador	4	24.36	2.32	0.58	97.45	0.024
maestro-trabajador	8	13.48	4.20	0.52	107.82	0.039

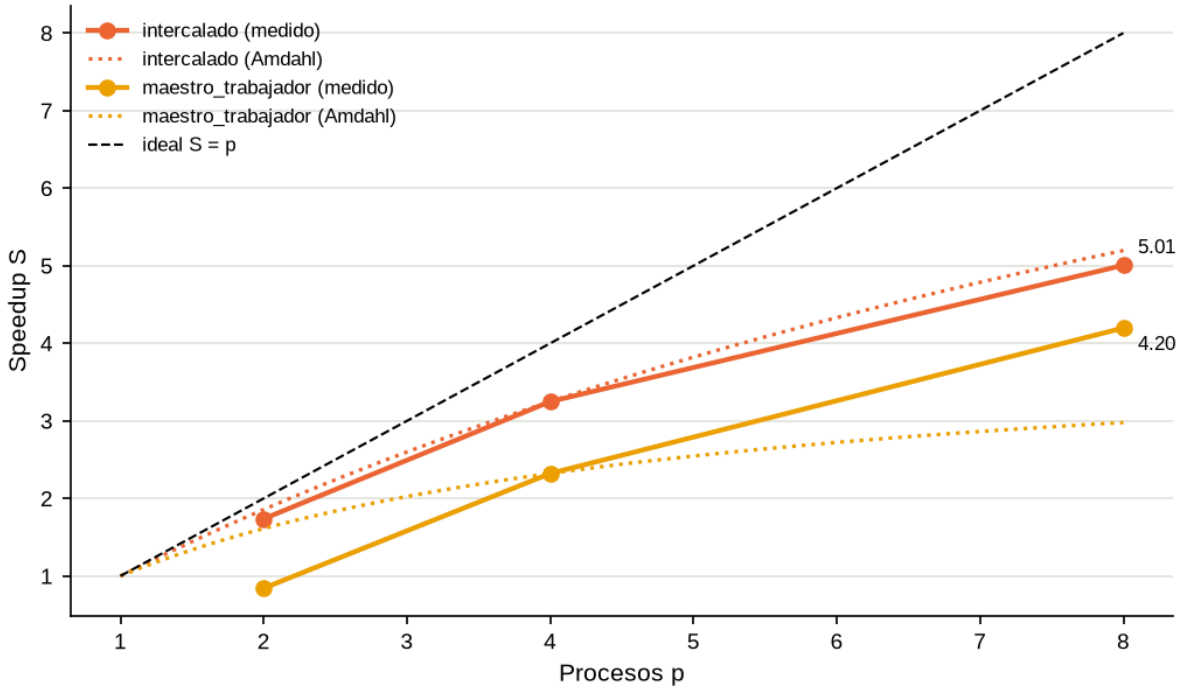


Figura 14. Speedup contra número de procesos con la llave media, con la recta ideal y la curva de Amdahl de cada programa.

Para Amdahl se toma el intercalado con p = 4: $f_p = (1 - 1/3.25) / (1 - 1/4) = 0.923$, así que $f_s = 0.077$ y el techo $1/f_s$ ronda 13. Con esa fracción, Amdahl predice 5.20 para p = 8 y se midieron 5.01, un 3.7 % menos; para p = 2 predice 1.86 y se midieron 1.74. Esa f_s no es código secuencial: es la pérdida de frecuencia al activar más núcleos, que crece con p en vez de quedarse fija. Por eso Amdahl, calibrado en p = 4, sobrestima los extremos.

El maestro-trabajador no sigue a Amdahl, porque su pérdida es estructural: con p procesos tiene p − 1 trabajadores. Con p = 2 hay uno solo y es más lento que el secuencial (S = 0.84); con p = 8 el maestro pesa 1/8 y el speedup sube a 4.20, muy por encima de los 2.98 que predice Amdahl con la f_s de p = 4.

La ley de Gustafson describe mejor la fuerza bruta, porque el problema crece solo: con S ≈ 3.25 se cubren 3.25 veces más llaves en el mismo tiempo, es decir, log2(3.25) ≈ 1.7 bits más, y con el híbrido de 16 trabajadores (S ≈ 10) unos 3.3 bits más. Al ritmo medido del secuencial, unos $2.97 \cdot 10^6$ llaves por segundo, recorrer las 2^{56} llaves tomaría unos 770 años; el intercalado con 4 procesos lo bajaría a unos 237 y el híbrido de 16 trabajadores a unos 79. Hacerlo en un día exigiría unos 281 000 núcleos con eficiencia 1.

La efectividad sigue creciendo hasta el p más grande que se midió: 0.055 con 8 procesos en el intercalado y 0.105 con el híbrido de 16 trabajadores. Todavía conviene agregar trabajadores, aunque la eficiencia baje de 0.87 a 0.63; si se exige una eficiencia de al menos 0.8, el punto adecuado es p = 4.

El híbrido frente a la versión sin OpenMP

La tabla da el speedup y la eficiencia del híbrido con 4 procesos y 1, 2 y 4 hilos por proceso. La figura 15 muestra el tiempo y el speedup contra los hilos.

Llave	h = 1 (w = 4)	h = 2 (w = 8)	h = 4 (w = 16)
fácil	S = 3.32 · E = 0.83	S = 5.69 · E = 0.71	S = 11.08 · E = 0.69
media	S = 3.37 · E = 0.84	S = 5.75 · E = 0.72	S = 9.75 · E = 0.61
difícil	S = 3.35 · E = 0.84	S = 5.91 · E = 0.74	S = 9.92 · E = 0.62

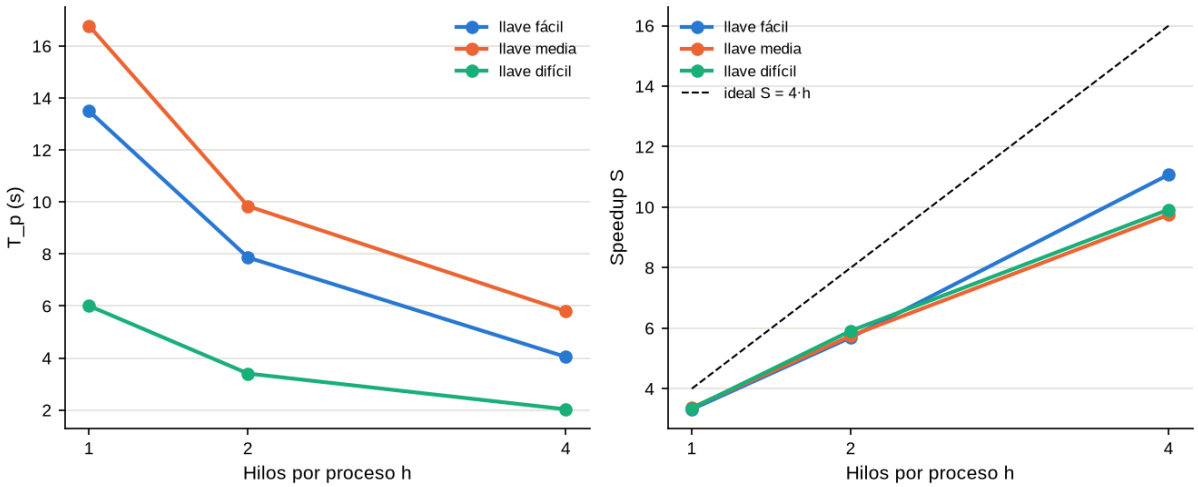


Figura 15. Tiempo y speedup del híbrido con 4 procesos según los hilos por proceso.

Con un hilo, el híbrido es el intercalado y tarda entre 0.96 y 0.98 veces lo mismo (13.50 s frente a 14.04 s con la llave fácil, 16.78 frente a 17.41 con la media y 6.01 frente a 6.14 con la difícil). Con más hilos, el tiempo baja de forma pareja en las tres llaves: el speedup ronda 5.7 a 5.9 con 2 hilos y 9.8 a 11.1 con 4, con S_{iter} exactamente 8 y 16 y S_{rend} sin superar w. Frente a `intercalado -np 8`, con los mismos 8 trabajadores pero todos MPI, el híbrido de 4 procesos con 2 hilos tarda 9.84 s contra 11.30 s, porque los hilos comparten el proceso y no hay ocho procesos sondeando MPI.

La eficiencia baja con 16 trabajadores porque el i9 tiene 8 núcleos de rendimiento: pasados 8 trabajadores, los demás comparten núcleo por *hyperthreading* o caen en núcleos de eficiencia, y como el reparto es estático, todos esperan al más lento. Aun así, el speedup crece de forma consistente con w.

Conclusiones y recomendaciones

Conclusiones

El reparto decide el speedup, no el número de procesos. Con el mismo código de búsqueda y los mismos 4 procesos, el naive va de S = 0.79 con la llave difícil a S = 11 000 aparente con la fácil, mientras el intercalado y el bloque-cíclico dan de 3.19 a 3.28 con cualquier llave. El tiempo del naive depende de dónde cae la llave dentro del bloque de quien la tiene; el de los repartos alternativos solo depende de k/p.

Una sola llave no basta para comparar algoritmos. Con k = 60 000 000 y 26 bits, el naive le ganó al intercalado (3.71 s frente a 5.87 s) porque esa llave cae en el último cuarto del espacio, donde el naive gana siempre; con una llave del primer cuarto pasa lo contrario. Sobre todas las llaves, cada uno gana en la mitad y en promedio empatan, y con 16 llaves al azar el speedup del naive fue de 0.83 a 7.83 contra 3.00 a 3.53 del intercalado. Lo que gana un enfoque alternativo es consistencia, no un mejor promedio.

El proyecto no es superlineal. Los S mayores que p del naive son un efecto de búsqueda: el proceso que tiene la llave empieza cerca de ella, pero no prueba llaves más rápido, y el speedup de rendimiento nunca pasó de p en ningún programa. El maestro-trabajador queda en S ≈ 2.3 con 4 procesos, 0.77 de su techo de p − 1 = 3 porque el maestro no busca, y a cambio balancea solo la carga cuando los núcleos son desiguales.

La parte secuencial de Amdahl no es código secuencial, sino pérdida efectiva: el turbo que baja con más núcleos activos, los núcleos de eficiencia, el sondeo de MPI y, en el maestro-trabajador, el propio maestro. Por eso Amdahl, calibrado con un valor de p, no predice bien los demás. La fuerza bruta escala al estilo de Gustafson: con w trabajadores se cubren log2(w) bits más en el mismo tiempo, aunque las 2^{56} llaves seguirían tomando unos 770 años en un núcleo.

El híbrido MPI + OpenMP es consistente con la versión sin OpenMP: con un hilo tarda de 0.96 a 0.98 del intercalado, y con 2 y 4 hilos el speedup sube a cerca de 5.8 y de 10, parejo en las tres llaves. Con 8 trabajadores queda algo mejor que el intercalado puro, siempre que los procesos no queden atados a un núcleo.

Retos y cómo se resolvieron

Reto	Solución
`rpc/des_crypt.h` ya no existe en glibc moderna	DES de OpenSSL (`DES_ecb_encrypt`), verificado contra `openssl enc -des-ecb`
Errores impresos por todos los procesos y procesos colgados	Solo el proceso 0 lee y valida; difunde un encabezado de estado con `MPI_Bcast` y todos salen limpios
El aviso de llave no llegaba mientras se calculaba	`MPI_Irecv` antes de buscar y `MPI_Test` cada 4096 llaves, que hace avanzar la recepción

Reto	Solución
La prueba b con 56 bits no termina (unos 150 años)	Límite `-t` con un centinela que el proceso 0 difunde al vencer, y la misma prueba con 28 bits
Medir con 56 bits es imposible	Presets que escalan con $N = 2^{bits}$: la llave conserva su posición relativa y S_{iter}
El maestro-trabajador se colgaba si los chunks se acababan antes de hallar la llave	Protocolo con dos invariantes: una respuesta por pedido y un único mensaje final por trabajador
El híbrido no escalaba con más hilos	Open MPI 5 ata cada proceso a un núcleo, así que se lanza con `--bind-to none`
Varios hilos con un MPI sin soporte multihilo completo	`MPI_THREAD_FUNNELED`: solo el hilo principal llama a MPI
Una comparación con una sola llave favorecía al naive	Fórmulas de intentos por programa, prueba con 16 llaves al azar y un visualizador que muestra qué llaves prueba cada proceso

Aprendizajes

Un speedup aislado no dice nada sin el caso que se midió. Hay que reportar la llave, los bits, el T_s del secuencial real y la mediana de varias corridas, y comparar con varias llaves. El speedup por iteraciones y el de rendimiento separan lo que aporta el reparto de lo que aporta el hardware y la suerte de dónde cae la llave.

Cuando la comunicación es mínima, los detalles del lanzador y del hardware pesan tanto como el algoritmo: el *binding* de `mpirun`, el turbo y la mezcla de núcleos de rendimiento y de eficiencia cambiaron los resultados más que el propio reparto. Y un protocolo de mensajes necesita invariantes explícitas, porque los casos borde, como tener pocos chunks o hallar la llave al final, son justo los que no aparecen con los parámetros por defecto.

Recomendaciones

Para fuerza bruta con llaves de distribución desconocida conviene un reparto intercalado o bloque-cíclico, que dan el speedup esperado para cualquier llave sin comunicación extra. Si los núcleos son desiguales o la máquina es compartida, conviene el maestro-trabajador, siempre que haya suficientes procesos para que el maestro pese poco.

Al medir, conviene usar el secuencial real con las mismas banderas de compilación, la máquina sin carga, al menos 5 repeticiones y varias llaves repartidas por todo el espacio. Cualquier $S > p$ debe tratarse como un error hasta explicarlo: revisar que T_s sea de la misma llave y bits, que el reparto no se salte llaves (S_{iter} igual a la teoría) y que T_p no se mida después del aviso. En los programas híbridos hay que revisar el *binding* con `mpirun --report-bindings` y no pedir más hilos que núcleos de rendimiento.

Para atacar los 56 bits reales, el paso siguiente es un clúster o el uso de aceleradores como las GPU: el problema es vergonzosamente paralelo y escala con el número de núcleos.

Referencias

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *Proceedings of the AFIPS Spring Joint Computer Conference* (pp. 483–485). ACM. https://doi.org/10.1145/1465482.1465560

Barlas, G. (2015). *Multicore and GPU programming: An integrated approach*. Morgan Kaufmann.

Electronic Frontier Foundation. (1998). *Cracking DES: Secrets of encryption research, wiretap politics & chip design*. O'Reilly.

Fedora Project. (s. f.). *Changes/SunRPCRemoval: Removal of Sun RPC interfaces from glibc*. Fedora Project Wiki. https://fedoraproject.org/wiki/Changes/SunRPCRemoval

Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM, 31*(5), 532–533. https://doi.org/10.1145/42411.42415

ISO/IEC. (2011). *ISO/IEC 9899:2011. Information technology — Programming languages — C*.

Message Passing Interface Forum. (2023). *MPI: A message-passing interface standard (Version 4.1)*. https://www.mpi-forum.org/docs/

National Institute of Standards and Technology. (1999). *FIPS PUB 46-3: Data Encryption Standard (DES)*. U.S. Department of Commerce.

National Institute of Standards and Technology. (2005). Announcing approval of the withdrawal of Federal Information Processing Standard (FIPS) 46-3, Data Encryption Standard (DES). *Federal Register, 70*(96), 28907–28908.

Open MPI Project. (s. f.). *mpirun / prterun: Mapping, ranking, and binding* (Open MPI 5.0.x documentation). https://docs.open-mpi.org/

OpenMP Architecture Review Board. (2021). *OpenMP application programming interface (Version 5.2)*. https://www.openmp.org/specifications/

OpenSSL Project. (s. f.). *DES_random_key, DES_set_key, DES_ecb_encrypt: DES encryption* (OpenSSL 3 manual pages). https://docs.openssl.org/3.0/man3/DES_random_key/

Pacheco, P. (2011). *An introduction to parallel programming*. Morgan Kaufmann.

Stallings, W. (2020). *Cryptography and network security: Principles and practice* (8.ª ed.). Pearson.

Weimer, F. (2020, julio). *[PATCH] Remove --enable-obsolete-rpc configure flag* [Mensaje en lista de correo]. libc-alpha. https://inbox.sourceware.org/libc-alpha/20200711181508.GA9224@x230/t

Anexo 1: Catálogo de funciones y librerías

Todas las funciones del proyecto, incluidas las `static`, agrupadas por archivo, con sus entradas, salidas y una descripción. Al final están las funciones de librería que se usaron: OpenSSL, Open MPI, la libc, OpenMP y los módulos de Python.

`des_utils.c`: DES sobre OpenSSL

Función	Entradas	Salidas	Descripción
`build_des_key`	`uint64_t key56`; `unsigned char out[8]`	`out`: 8 bytes con 7 bits de llave y 1 de paridad	Reparte los 56 bits en grupos de 7 (bits 1 a 7 de cada byte) y completa la paridad impar con `DES_set_odd_parity`, igual que el `decrypt()` original
`run_des` (`static`)	`uint64_t key`; `unsigned char *buffer`; `size_t length` (múltiplo de 8); `int direction`	`buffer` transformado en el mismo lugar	Arma la llave, genera las subllaves con `DES_set_key_unchecked` (sin rechazar llaves débiles) y aplica `DES_ecb_encrypt` bloque a bloque
`encrypt_des`, `decrypt_des`	`uint64_t key`; `unsigned char *buffer`; `size_t length`	`buffer` cifrado o descifrado	Llaman a `run_des` con `DES_ENCRYPT` o `DES_DECRYPT`; reemplazan a `encrypt` y `decrypt` del `bruteforce.c`
`try_key`	`uint64_t key`; `const unsigned char *cipher`; `size_t length`; `const char *phrase`; `unsigned char *workspace` (`length + 1` bytes)	`int`: 1 si el descifrado contiene la frase	Copia el cifrado al `workspace` con `memcpy`, lo descifra con la llave candidata y busca la frase con `strstr`
`padded_length`	`size_t length`	`size_t`	Redondea al siguiente múltiplo de 8 (relleno de ceros hasta completar bloques DES)

`options.c`: Parámetros

Función	Entradas	Salidas	Descripción
`print_usage` (`static`)	`const char *program`; `int flags`	ayuda en stderr	Forma de uso; `-c`, `-o` y `-v` solo aparecen en los programas que las admiten
`parse_u64` (`static`)	`const char *text`; `uint64_t *value`	`int`: 0 si la conversión fue completa; `*value`	Envuelve `strtoull`; acepta decimal, `0x` y una `L` final, y rechaza basura, signo o desbordamiento

Función	Entradas	Salidas	Descripción
`resolve_key`	`const char *text`; `int key_bits`; `uint64_t *key_out`	`int`: 0 si es válida; `*key_out`	Traduce los presets `a`, `b`, `c`, `facil`, `media` y `dificil` (escalados con $N = 2^{bits}$) o convierte el número, y valida que sea menor que N
`parse_options`	`argc`, `argv`; `const char *program`; `int flags` (`OPT_OUTPUT`, `OPT_CHUNK`, `OPT_PROGRESS`); `program_options_t *opts`	`OPTIONS_OK`, `OPTIONS_HELP` u `OPTIONS_ERROR`; `*opts`	Lee las opciones con `getopt` y valida los rangos de `-b`, `-t` y `-c`, que `-c` sea menor que `-b`, la frase y la llave; `-v` activa el reporte de avance

`problem.c`: Problema y reporte

Función	Entradas	Salidas	Descripción
`read_text_file` (`static`)	`const char *path`; `size_t *length_out`	texto terminado en `'\0'` o `NULL`; bytes útiles	Lee el archivo completo, hasta 64 KiB; quita saltos de línea finales y rechaza archivos vacíos, grandes o con bytes nulos
`problem_prepare`	`const program_options_t *opts`; `crack_problem_t *problem`	`int`: 0 si todo salió bien; `*problem`	Carga el texto, verifica que contenga la frase clave y lo cifra con la llave privada
`problem_release`	`crack_problem_t *problem`	punteros liberados y en `NULL`	Libera texto, cifrado y frase; se puede llamar más de una vez
`problem_print_header`	programa, `opts`, `problem`, `int processes`	encabezado en stdout	Archivo, frase, llave y tamaño del espacio antes de buscar
`problem_print_result`	programa, `opts`, `problem`, `const search_result_t *result`	reporte y línea `CSV`, de 11 columnas	Llave hallada, quién la halló y tras cuántos intentos, texto descifrado, llaves probadas, ritmo y tiempo
`print_hex`	`const char *label`; bytes; `size_t length`	hexadecimal en stdout	Imprime un buffer en grupos de 8 bytes
`wall_seconds`	—	`double`	Reloj de pared monotónico (`clock_gettime(CLOCK_MONOTONIC)`)
`print_progress`	`int worker`; `uint64_t iterations`; `double elapsed_s`	línea `PROGRESS`,	Avance de un trabajador para el visualizador (solo con `-v`)

Función	Entradas	Salidas	Descripción
`print_chunk`	`int worker`; `uint64_t first_key`; `double elapsed_s`	línea `CHUNK,`	Chunk que empieza un trabajador del maestro-trabajador (solo con `-v`)

`cifrar.c` y `secuencial.c`: Partes B.1 y A.3

Función	Entradas	Salidas	Descripción
`write_cipher_file` (`static`)	`const char *path`; bytes; `size_t length`	`int`: 0 si se escribió completo	Guarda el cifrado en binario y revisa `fwrite` y `fclose`
`main` (cifrar)	`-f`, `-k`, `-p`, `-b`, `-o`	`EXIT_SUCCESS` si el descifrado coincide; texto, cifrado en hexadecimal y contraejemplo	Cifra el `.txt`, lo descifra de vuelta, lo compara con el original y muestra qué sale con la llave vecina
`main` (secuencial)	`-f`, `-k`, `-p`, `-b`, `-t`, `-v`	`EXIT_SUCCESS` si la halló o si se usó `-t`; reporte y CSV	Recorre las llaves 0, 1, 2, … hasta que `try_key` acierta o vence el límite, que revisa cada 4096 llaves

`search_mpi.c`: Módulo común MPI

Función	Entradas	Salidas	Descripción
`validate_processes` (`static`)	`int size`; `int min_processes`; `problem`	`int`: 0 si sirve	El proceso 0 rechaza menos procesos que el mínimo (el maestro-trabajador pide 2) o más procesos que llaves
`share_problem` (`static`)	`mpi_search_t *search`; `int status`	estado del proceso 0, igual en todos	Difunde con `MPI_Bcast` un encabezado de 7 enteros (estado, bits, longitudes, límite, chunk y `-v`) y luego el cifrado y la frase; si el estado no es «listo», todos salen sin quedar esperando
`mpi_search_setup`	`argc`, `argv`; programa; `int option_flags`; `int min_processes`; `search`	`SEARCH_READY`, `SEARCH_HELP` o `SEARCH_ERROR`; `*search`	Inicia MPI si no lo está (`MPI_Initialized`); el proceso 0 lee parámetros y archivo y valida los procesos; se reparte el problema y cada proceso reserva su `workspace`
`mpi_search_release`	`search`	memoria liberada y MPI cerrado	Libera el `workspace` y el problema y llama a `MPI_Finalize`

Función	Entradas	Salidas	Descripción
`mpi_search_begin`	—	`double`: `MPI_Wtime` tras la barrera	Sincroniza con `MPI_Barrier` y marca el inicio de la búsqueda
`notify_all` (`static`)	`const uint64_t message[3]`; `int size`	un `MPI_Send` a cada proceso	Envía el aviso a todos, incluido uno mismo, para que todos los `MPI_Irecv` se completen
`announce_timeout` (`static`)	`found_listener_t *listener`; `int size`	`timeout_sent` en 1	Avisa con la llave centinela `UINT64_MAX` que venció el límite
`found_listener_start`	`listener`; `search`	antena puesta, reloj y plazo	Pone el `MPI_Irecv` antes de buscar, sincroniza y arranca el reloj; el proceso 0 fija el plazo
`found_listener_poll`	`listener`; `search`	`int`: 1 si ya llegó un aviso	Revisa la antena con `MPI_Test`; el proceso 0 además avisa si venció el plazo
`found_listener_announce`	`search`; `uint64_t key`; `uint64_t iterations`	aviso enviado	Arma {llave, rango, iteraciones} y lo envía con `notify_all`
`found_listener_progress`	`listener`; `search`; `int worker`; `uint64_t iterations`	línea `PROGRESS,`	Reporta el avance del proceso si se pidió `-v`
`found_listener_try_key`	`listener`; `search`; `uint64_t key`; `uint64_t *iterations`	`int`: 1 si hay que parar; contador incrementado	Un paso del bucle estático: sondea cada 4096 llaves (y reporta avance con `-v`), prueba la llave y, si acierta, avisa a todos
`found_listener_finish`	`listener`; `search`; `search_result_t *result`	`*result`: hallazgo, llave, proceso, iteraciones y tiempo	El proceso 0 vigila el plazo con pausas de 1 ms; luego todos cierran la antena con `MPI_Wait` y toman el tiempo
`mpi_search_report`	`search`; programa; `result`; `uint64_t own_iterations`	reporte y CSV del proceso 0	Suma las llaves de todos con `MPI_Reduce` y el proceso 0 llama a `problem_print_result`

Programas MPI e híbrido

Función	Entradas	Salidas	Descripción
`main` (naive, intercalado, ciclico)	`-f`, `-k`, `-p`, `-b`, `-t`, `-v` (y `-c` en ciclico)	`EXIT_SUCCESS` o `EXIT_FAILURE`; reporte y CSV	`mpi_search_setup` → `found_listener_start` → bucle con `found_listener_try_key` → `found_listener_finish` → `mpi_search_report`. Solo cambia el recorrido: bloque `[r·N/p, (r+1)·N/p)`, llaves `r, r+p, …` o chunks `r, r+p, …` de 2^c llaves
`run_master` (`static`)	`search`; `search_result_t *result`	`*result` con hallazgo, proceso, iteraciones y T_p	Atiende pedidos con `MPI_Recv(ANY_SOURCE)` y responde a cada uno un chunk (`TAG_WORK`) o `TAG_STOP`; responde STOP desde el hallazgo (ahí toma T_p), el plazo o el fin de los chunks, y termina al recibir el mensaje final de cada trabajador
`search_chunk` (`static`)	`search`; `const uint64_t work[2]`; `double start`; `uint64_t *iterations`; `uint64_t *found_key`	`int`: 1 si la halló	Prueba en orden las llaves de un chunk; con `-v` reporta el chunk y el avance
`run_worker` (`static`)	`search`	`uint64_t`: llaves probadas	Pide el chunk siguiente por adelantado (`MPI_Send` + `MPI_Irecv`), prueba el actual y espera el próximo con `MPI_Wait`; termina con FOUND o DONE
`main` (maestro_trabajador)	`-f`, `-k`, `-p`, `-b`, `-t`, `-c`, `-v`	`EXIT_FAILURE` con `-np 1`; reporte y CSV	El proceso 0 corre `run_master` y los demás `run_worker`; el maestro aporta 0 llaves
`init_threaded_mpi` (`static`)	`argc`, `argv`	`int`: 0 si MPI da `MPI_THREAD_FUNNELED`	Inicia MPI con `MPI_Init_thread` y valida el nivel concedido
`share_threads` (`static`)	`search`	`int`: hilos por proceso T, o −1	El proceso 0 lee T con `omp_get_max_threads`, valida $1 \le T \le 256$ y $p \cdot T \le N$, y lo difunde con `MPI_Bcast`

Función	Entradas	Salidas	Descripción
`search_thread` (`static`)	`listener`; `search`; `int threads`; `thread_shared_t *shared`	`uint64_t`: llaves del hilo; hallazgo en `*shared`	El hilo t del proceso r prueba g, g + pT, … con g = r·T + t; cada 4096 llaves revisa la bandera atómica, el hilo 0 sondea la antena y, con `-v`, cada hilo reporta su avance
`main` (hibrido)	`-f`, `-k`, `-p`, `-b`, `-t`, `-v`; `OMP_NUM_THREADS`	reporte y CSV con `hilos = T`	Abre la región paralela con `reduction(+)` de las llaves y, fuera de ella, el hilo principal envía el aviso si un hilo halló la llave

`scripts/analizar.py`: Análisis de la campaña

Funciones de Python que convierten `resultados/campana_<host>_*.csv` en `resultados/analisis_<host>.md` y las gráficas de `docs/img/`.

Función	Entradas	Salidas	Descripción
`Group` (clase)	caso, programa, procesos, hilos, bits, llave real	objeto con listas de tiempos, iteraciones y llaves probadas	Repeticiones de un mismo caso; `time`, `finder_iterations` y `keys_tried` dan la mediana y `workers` = p·h
`case_rank`, `program_rank`	`str` caso o programa	`int`	Orden fijo de llaves y programas en tablas y gráficas
`host_from_paths`	`list[str]` rutas	`str` host	Saca el host del nombre `campana_<host>_<fecha>.csv`
`load_groups`	`list[str]` rutas	grupos válidos, todos los grupos, `int` filas	Lee los CSV, agrupa y aparta las corridas con `llave_hallada != llave_real`
`sequential_times`	grupos	`dict` (caso, bits) → T_s	Mediana del `secuencial` por llave y bits
`metrics`	`Group`, `float` T_s	`dict` S, E, C, F, S_{iter}, S_{rend}	Las fórmulas de clase con w = p·h
`amdahl_fp`	`float` S, `int` p	`float` f_p	$f_p = (1 - 1/S)/(1 - 1/p)$
`amdahl_speedup`	`float` f_p, `float` p	`float` S	$1/(f_s + f_p/p)$
`gustafson_speedup`	`float` f_s, `int` p	`float` S	$p - f_s(p - 1)$
`ideal_speedup`	`Group`	`int`	p·h, o p − 1 en el maestro-trabajador

Función	Entradas	Salidas	Descripción
`note`	`Group`, métricas, lista de alertas	`str` nota	Regla del superlineal: «aparente (búsqueda)» en el naive; `REVISAR` si otro programa da S > w o cualquiera da S_{rend} > w (tolerancia 2 %)
`fmt`	`float`, dígitos	`str`	Número legible, en notación científica si es enorme o diminuto
`time_cell`, `label`	`Group`	`str`	Celda «mediana (mín–máx)» y nombre del programa con sus hilos
`main_table`	líneas, grupos, T_s, alertas	líneas de Markdown	Tabla de T_p, S, E, C, F, S_{iter}, S_{rend} y nota
`amdahl_section`	líneas, grupos, T_s, p principal, llave del barrido	`dict` programa → f_p	f_p, f_s, techo, Gustafson y bits extra; predicción de Amdahl frente al barrido
`style_axes`	eje de matplotlib	—	Rejilla y ejes discretos
`plot_speedup_by_key`	matplotlib, grupos, T_s, p, ruta	PNG	Barras de S por programa y llave con la línea S = p; el naive aparente se recorta y se rotula
`plot_sweep`	matplotlib, grupos, T_s, f_p, llave, ruta	PNG	S contra p con la ideal y la curva de Amdahl
`plot_hybrid`	matplotlib, grupos, T_s, ruta	PNG	Tiempo y speedup del híbrido contra los hilos
`plot_all`	grupos, T_s, f_p, p, llave, host, carpeta	`list[str]` rutas	Genera las gráficas si hay matplotlib; si no, avisa y sigue
`analyze`	rutas, salida, carpeta de imágenes	`list[str]` alertas	Arma el `.md` completo e imprime las alertas
`parse_args`, `main`	línea de comandos	código de salida	1 si hubo alguna alerta `REVISAR`

`scripts/key_position_plots.py`: Figuras 12 y 13

Función	Entradas	Salidas	Descripción
`naive_tries`	`int` llave k	`int`	Intentos del proceso del naive que tiene k: $k - r\cdot(N/p) + 1$
`naive_wins`	`int` llave k	`bool`	Si el naive necesita menos intentos que el intercalado ($\lfloor k/p \rfloor + 1$)
`key_samples`, `zones`	—	lista de llaves; tramos (inicio, fin, gana el naive)	Puntos de la curva, incluidos los saltos de bloque, y zonas de fondo de la figura 12

Función	Entradas	Salidas	Descripción
`style_axes`	eje de matplotlib	—	Mismo estilo que `analizar.py`
`plot_tries`	ruta del PNG	`docs/img/intentos_por_llave.png`	Intentos de cada programa según la posición de la llave, con las llaves 60 000 000 y 16 000 000 marcadas
`load_campaign`	—	`dict` (llave, programa) → tiempo	Lee la campaña de 16 llaves al azar (híbrido con 2 hilos)
`plot_measured`	ruta del PNG	`docs/img/speedup_llaves_azar.png`	Speedup medido de cada programa según la posición de la llave
`main`	—	código de salida	Genera las dos figuras

`visualizer/`: Visualizador web

Función	Entradas	Salidas	Descripción
`bounded_int`	parámetros, nombre, mínimo, máximo	`int`	Valida que un parámetro sea un entero en rango; si no, lanza `BadRequest`
`build_command`	`dict` con programa, llave, bits, procesos, hilos, chunk y límite	comando, entorno y texto	Arma la llamada real (`mpirun ... -v`, con `--bind-to none` y `OMP_NUM_THREADS` en el híbrido); solo admite los seis programas
`parse_line`	`str` línea del programa	`dict`	Traduce las líneas `PROGRESS`, `CHUNK` y `CSV` a eventos JSON
`stop_process`	proceso	—	Termina el grupo de procesos de `mpirun`
`cpu_model`	—	`str`	Modelo de CPU para el encabezado de la página
`Handler` (clase)	peticiones HTTP	página, `/api/info`, `/api/run`, `/api/stop`	Solo atiende a `127.0.0.1`; corre un programa a la vez y transmite su avance línea por línea
`index.html`	eventos del servidor	rectángulo, lupa, carrera y tabla	Dibuja el espacio de llaves con un color por proceso, repite las corridas con el mismo reloj y compara tiempos y speedups

Librerías

OpenSSL (`<openssl/des.h>`, `-lcrypto`)

Función o tipo	Uso en el proyecto
`DES_cblock`	Bloque o llave de 8 bytes
`DES_key_schedule`	Subllaves de las 16 rondas

Función o tipo	Uso en el proyecto
`DES_set_odd_parity`	Completa la paridad impar de cada byte de la llave
`DES_set_key_unchecked`	Genera el *key schedule* sin rechazar llaves débiles
`DES_ecb_encrypt`	Cifra o descifra un bloque de 8 bytes en modo ECB

Open MPI (`<mpi.h>`)

Primitiva	Uso en el proyecto
`MPI_Init`, `MPI_Finalize`	Abren y cierran MPI (`mpi_search_setup` y `mpi_search_release`)
`MPI_Init_thread`	El híbrido inicia MPI pidiendo `MPI_THREAD_FUNNELED`
`MPI_Initialized`	`mpi_search_setup` no vuelve a iniciar MPI si el híbrido ya lo hizo
`MPI_Comm_rank`, `MPI_Comm_size`	Rango propio y número de procesos
`MPI_Bcast`	Reparte el encabezado, el cifrado y la frase desde el proceso 0
`MPI_Barrier`	Sincroniza el inicio de la búsqueda
`MPI_Wtime`	Reloj de pared para T_p y para el límite de tiempo
`MPI_Irecv`	Antena de aviso y pedido adelantado del maestro-trabajador
`MPI_Send`	Aviso de llave, pedidos y respuestas del maestro-trabajador
`MPI_Recv`	El maestro recibe pedidos; el trabajador recibe chunks y STOP
`MPI_Test`	Sondeo de la antena cada 4096 llaves (hace avanzar la recepción)
`MPI_Wait`	Cierra la antena y espera el chunk adelantado
`MPI_Reduce`	Suma de las llaves probadas por todos
`MPI_Abort`	Solo si falta memoria en un proceso

libc

Función	Uso en el proyecto
`memcpy`	Copia del cifrado al `workspace` en cada intento
`memset`, `memchr`	Inicializar estructuras; detectar bytes nulos en el `.txt`
`strstr`	Buscar la frase clave en el texto descifrado
`strlen`, `strcmp`, `strcat`, `strdup`	Frase, presets de llave, optstring y copia de la frase
`getopt`	Leer las opciones de la línea de comandos
`strtoull`	Convertir números de 64 bits (`parse_u64`)
`malloc`, `calloc`, `free`	Memoria del problema y del `workspace` (`calloc` deja el relleno en ceros)
`fopen`, `fread`, `fwrite`, `fclose`, `ferror`	Leer el `.txt` y guardar el cifrado
`printf`, `fprintf`, `snprintf`, `fflush`, `perror`, `strerror`	Reportes y mensajes de error

Función	Uso en el proyecto
`clock_gettime`	Reloj monotónico del secuencial
`nanosleep`	Pausa de 1 ms del proceso 0 mientras vigila el límite

OpenMP y C11 (híbrido)

Primitiva	Uso en el proyecto
`#pragma omp parallel num_threads(T) reduction(+ : iterations)`	Región paralela con T hilos; suma las llaves probadas de todos los hilos
`#pragma omp critical(found_key)`	Guarda llave e iteraciones del primer hilo que la halla
`omp_get_max_threads`	El proceso 0 lee T de `OMP_NUM_THREADS`
`omp_get_thread_num`	Número de hilo t, para el trabajador global g = r·T + t
`atomic_int`, `atomic_init`, `atomic_store`, `atomic_load_explicit` (`<stdatomic.h>`)	Bandera de parada compartida entre los hilos de un proceso

Python (análisis)

Módulo	Uso
`csv`, `statistics.median`	Leer la campaña y tomar la mediana de las repeticiones
`math.log2`	Bits extra según Gustafson
`argparse`, `os`, `re`, `sys`	Línea de comandos, rutas y alertas
`matplotlib` (opcional)	Gráficas en `docs/img/`

Anexo 2: Bitácora de pruebas y speedups

Máquina y software

Todas las mediciones de este anexo, la Parte A.3 y el análisis de speedup se tomaron en la laptop de Esteban, que es la máquina oficial del informe. Las pruebas B.1 y B.2 se hicieron en la MacBook Pro de Nico, como se indica en su capítulo. La tabla describe la máquina oficial.

Componente	Detalle
CPU	Intel Core i9-13980HX: 24 núcleos (8 de rendimiento con *hyperthreading* y 16 de eficiencia), 32 hilos
Memoria	16 GB
Sistema	CachyOS (Linux 7.2.9)
Compilador	GCC 16.2.1 con `-O2 -march=native -std=c11`, las mismas banderas en todos los programas
MPI y librerías	Open MPI 5.0.11, OpenSSL 3.6.5 y el OpenMP de GCC

La ficha completa, con la salida de `lscpu` y la versión de `mpirun`, queda en `resultados/maquina_escu.txt`.

Procedimiento

La campaña se corrió con los comandos siguientes, en este orden.

```
make clean && make
scripts/verificar.sh                                  # 126 casos, 0 fallas
OUT=resultados/secuencial_longitudes_escu.csv REPS=5 scripts/medir_longitudes.sh   #
Parte A.3
BITS=28 REPS=5 NP=4 NP_LIST="2 4 8" HYBRID_THREADS="1 2 4" scripts/campana.sh
python3 scripts/analizar.py resultados/campana_escu_<fecha>.csv
python3 scripts/capturas.py                           # capturas de este anexo
BITS=26 REPS=1 NP_LIST=4 HYBRID_THREADS="1 2" KEYS="<16 llaves al azar>"
scripts/campana.sh
python3 scripts/key_position_plots.py                 # figuras 12 y 13
```

Se usaron 28 bits, es decir, $N = 2^{28} = 268\,435\,456$ llaves. Los presets escalan con N, así que cada llave conserva la posición relativa que tiene con 56 bits. La llave fácil vale $N/2 + 1 = 134\,217\,729$, la media $N/2 + N/8 = 167\,772\,160$ y la difícil $\lceil N/7 + N/13 \rceil = 58\,996\,804$.

Cada caso se repitió 5 veces y se reporta la mediana, junto con el mínimo y el máximo. El tiempo es de pared. En los programas MPI se mide con `MPI_Wtime` desde la barrera hasta que llega el aviso, y en el secuencial con `clock_gettime(CLOCK_MONOTONIC)` alrededor de la búsqueda. T_s es la mediana del programa `secuencial` con la misma llave y los mismos bits, no el paralelo con un proceso. El híbrido se lanzó con `mpirun --bind-to none -x OMP_NUM_THREADS -np 4` y 1, 2 y 4 hilos.

Verificación

El script `scripts/verificar.sh` usa `-b 20` y corre cada programa con las 6 llaves (a, b, c, fácil, media y difícil) y con 2, 3 y 4 procesos. El híbrido se prueba además con 1 y 2 hilos. El maestro-trabajador se prueba con pocos chunks (`-b 18`), que se agotan antes de hallar la llave, y con un solo proceso, donde debe dar error sin colgarse. El resultado fue de 126 casos y 0 fallas, como muestra la figura 16.

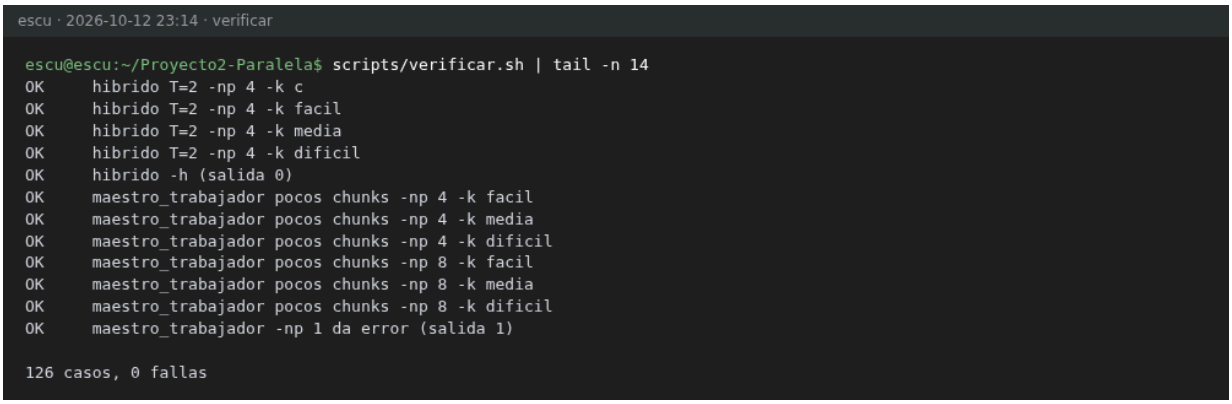


Figura 16. Final de la salida de `scripts/verificar.sh`, con los últimos casos y el resumen.

Campaña: tiempos (mediana de 5, en segundos)

La campaña tuvo 140 corridas y las 140 hallaron la llave real; el análisis no levantó ninguna alerta. El mínimo y el máximo de cada caso están en `resultados/analisis_escu.md`, y los speedups, el barrido de procesos y el híbrido se analizan en el capítulo de resultados. Las corridas de las pruebas B.1 y B.2 están en su capítulo y en `resultados/pruebas_b2.csv`.

Programa	p	h	fácil	media	difícil
secuencial (T_s)	1	1	44.792	56.569	20.127
naive	4	1	0.004	14.326	25.321
intercalado	4	1	14.040	17.414	6.136
ciclico (C = 2^{16})	4	1	13.942	17.645	6.152
maestro_trabajador (C = 2^{16})	4	1	19.472	24.362	8.563
hibrido	4	1	13.499	16.783	6.009
hibrido	4	2	7.868	9.842	3.404
hibrido	4	4	4.044	5.801	2.029

Capturas

Cada captura muestra la salida real de los comandos, tal como la imprimen los programas. Se tomaron con `scripts/capturas.py` en la misma máquina, después de la campaña. Los tiempos de una corrida suelta varían unas décimas respecto a la mediana.

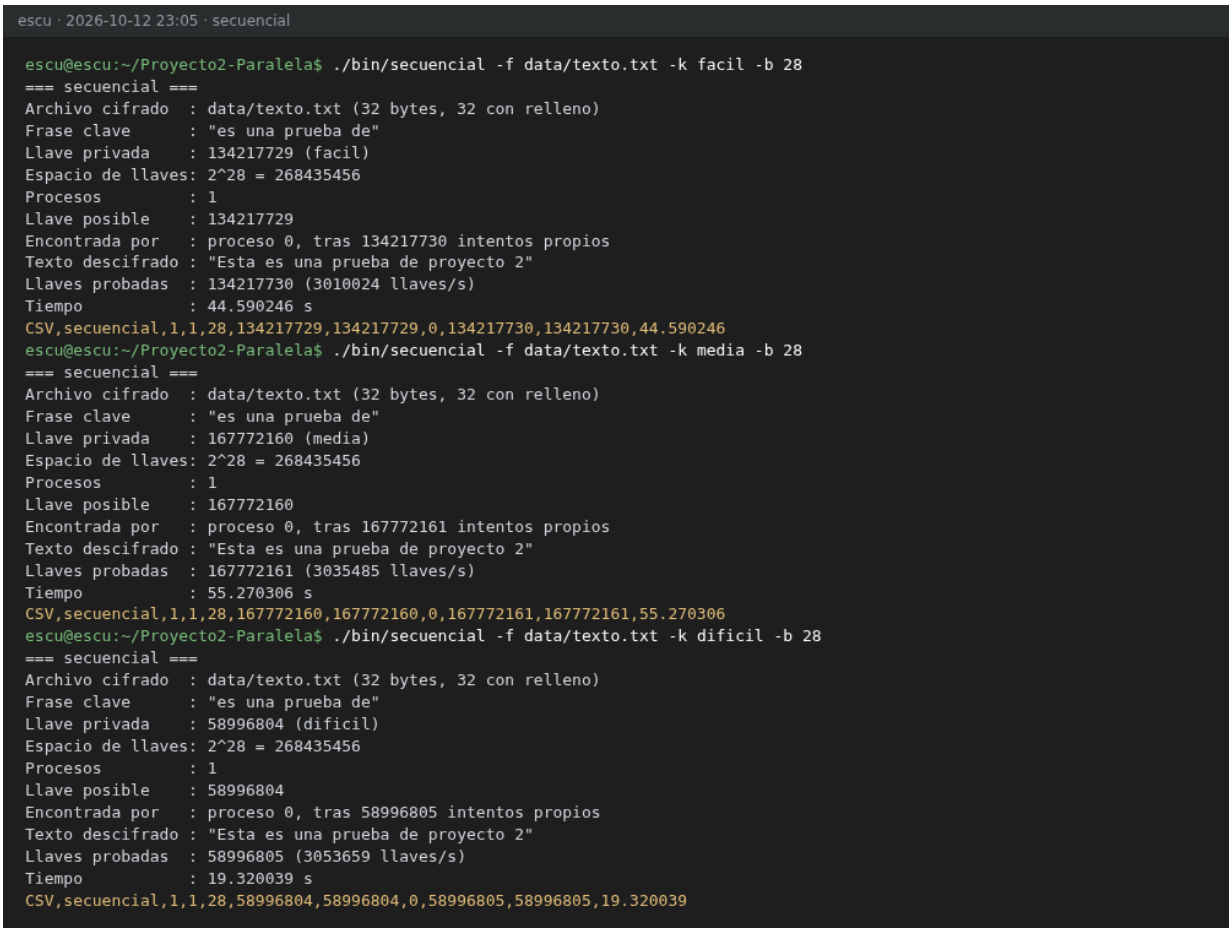


Figura 17. Secuencial con las llaves fácil, media y difícil; estas corridas dan el T_s.

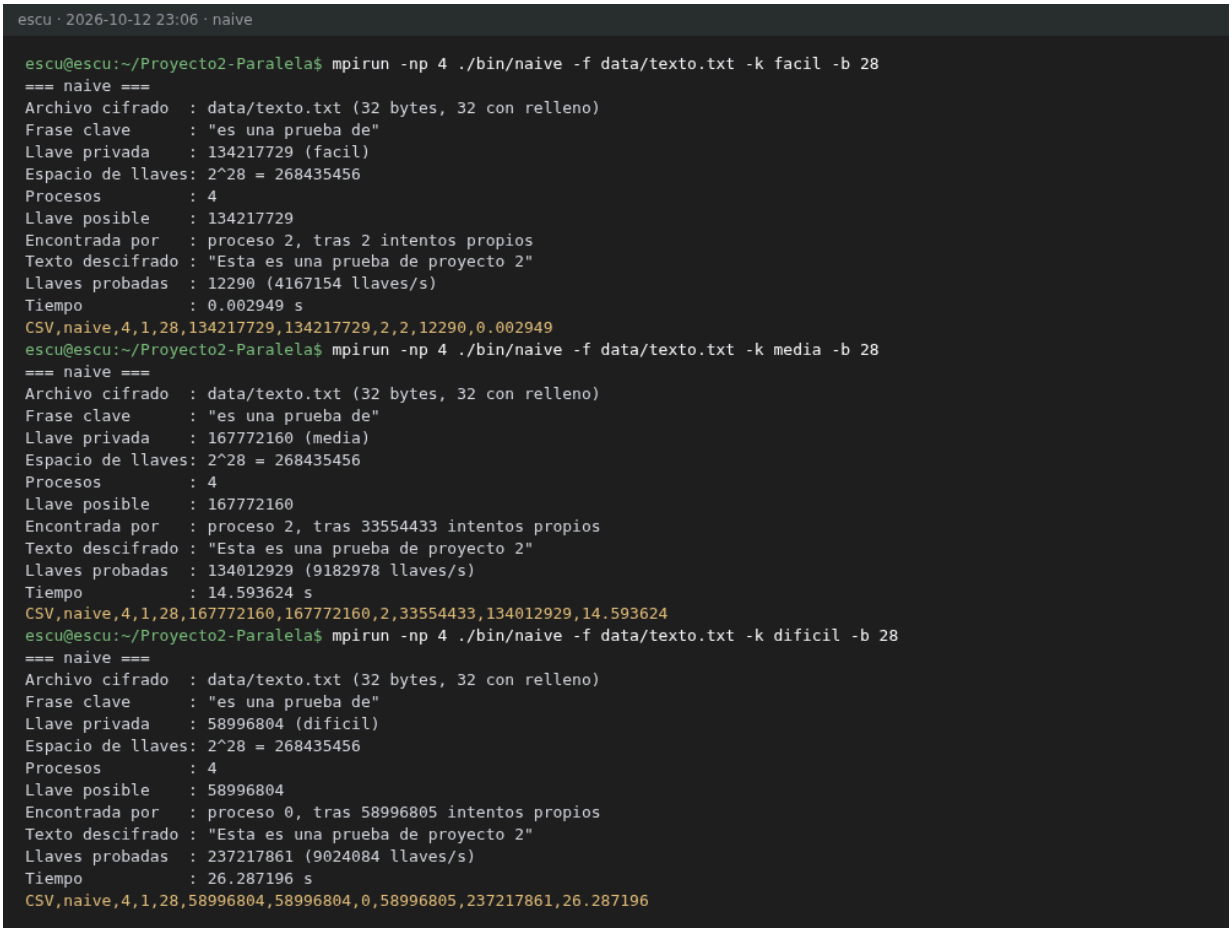


Figura 18. Naive con 4 procesos. Con la llave fácil, el proceso 2 la halla al segundo intento; con la media, a la mitad de su bloque; con la difícil, el proceso 0 la halla con las mismas iteraciones que el secuencial.

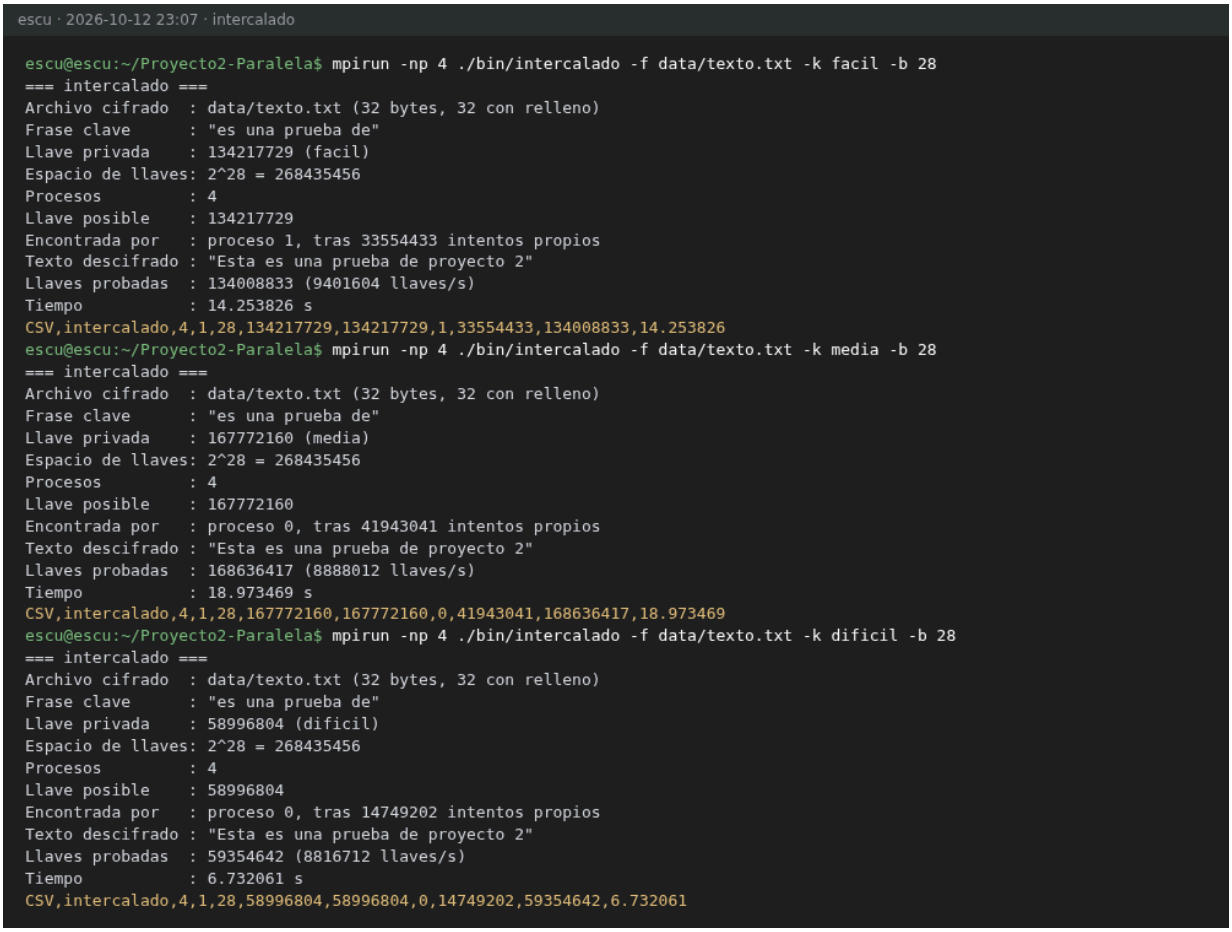


Figura 19. Intercalado con 4 procesos y las tres llaves.

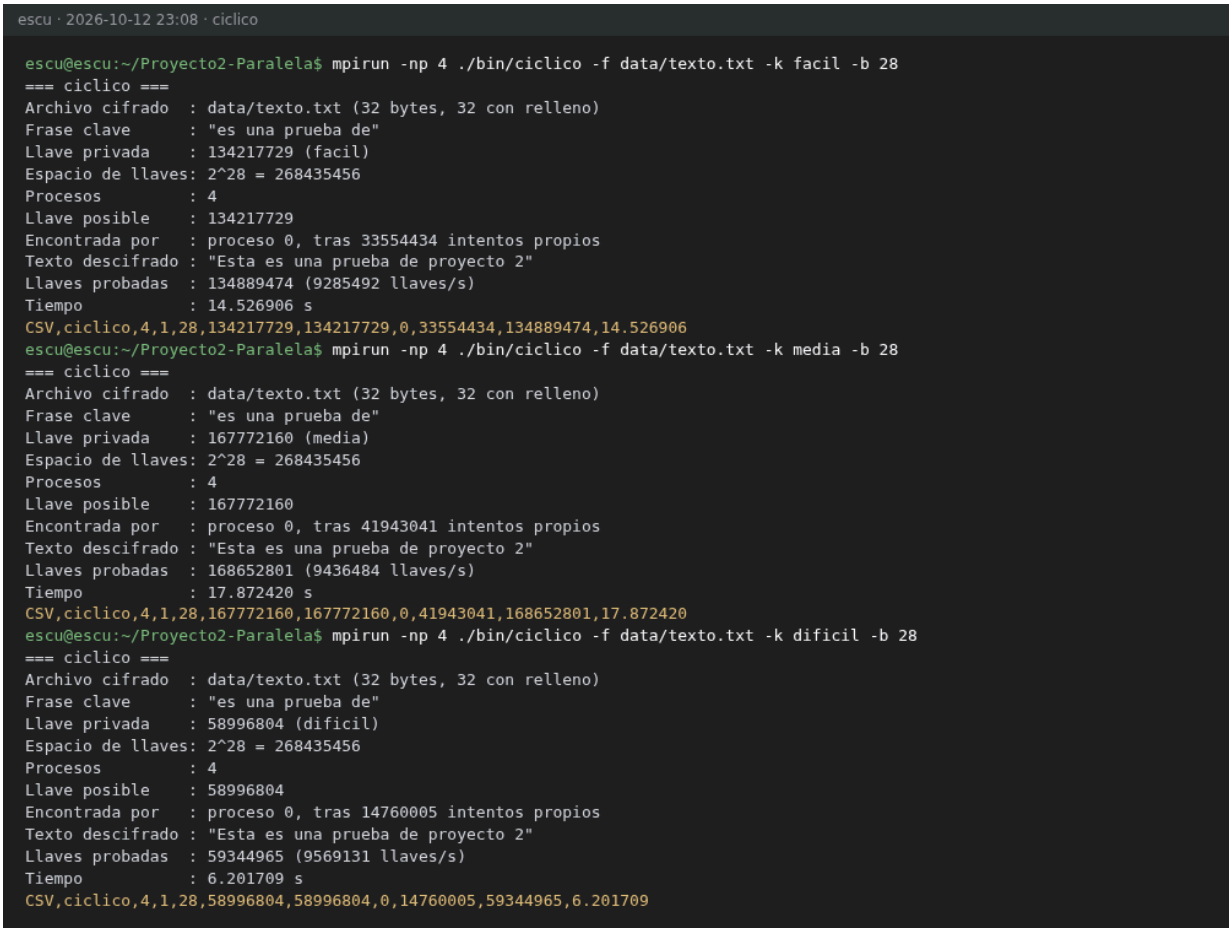


Figura 20. Bloque-cíclico con 4 procesos y chunks de 2^{16} llaves.

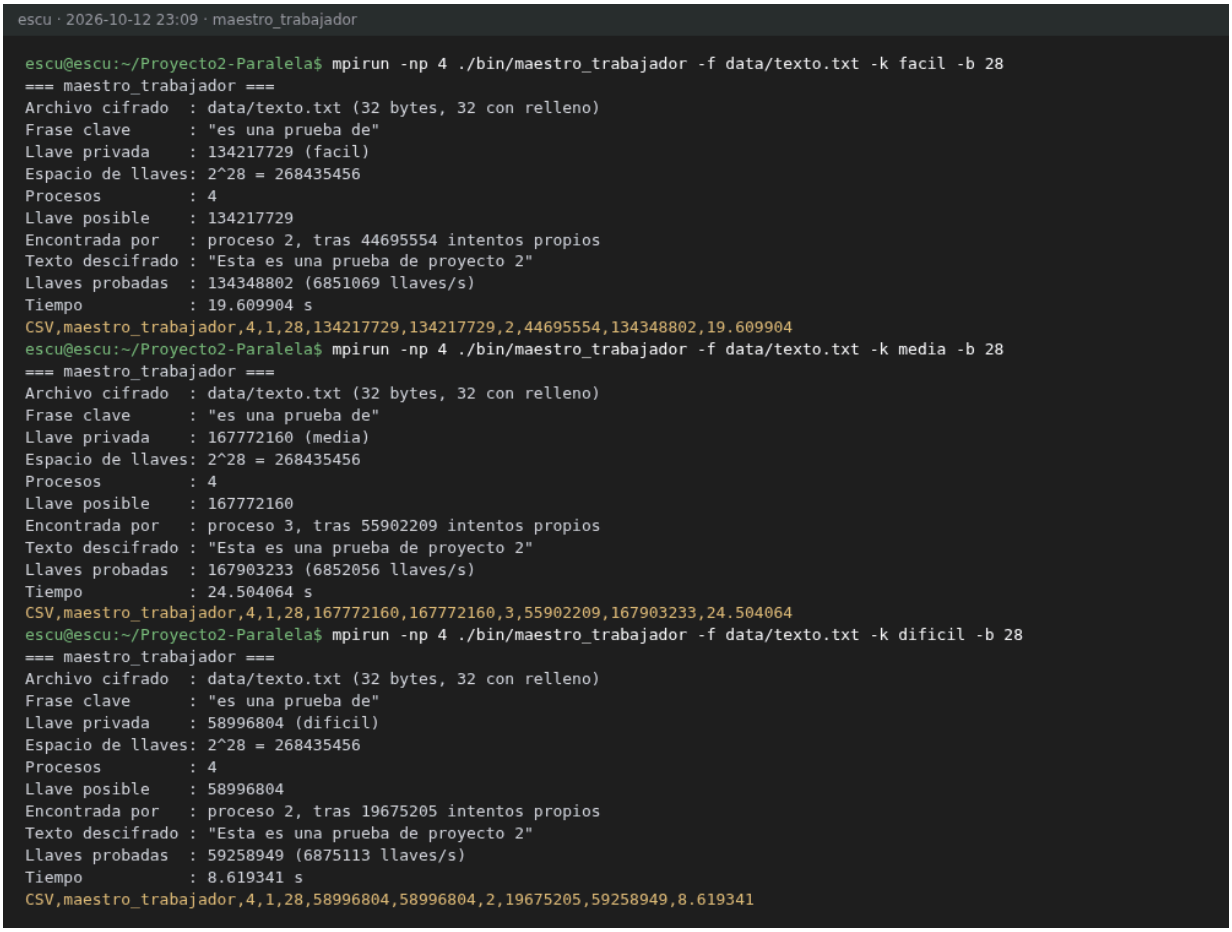


Figura 21. Maestro-trabajador con 4 procesos, es decir, un maestro y tres trabajadores.

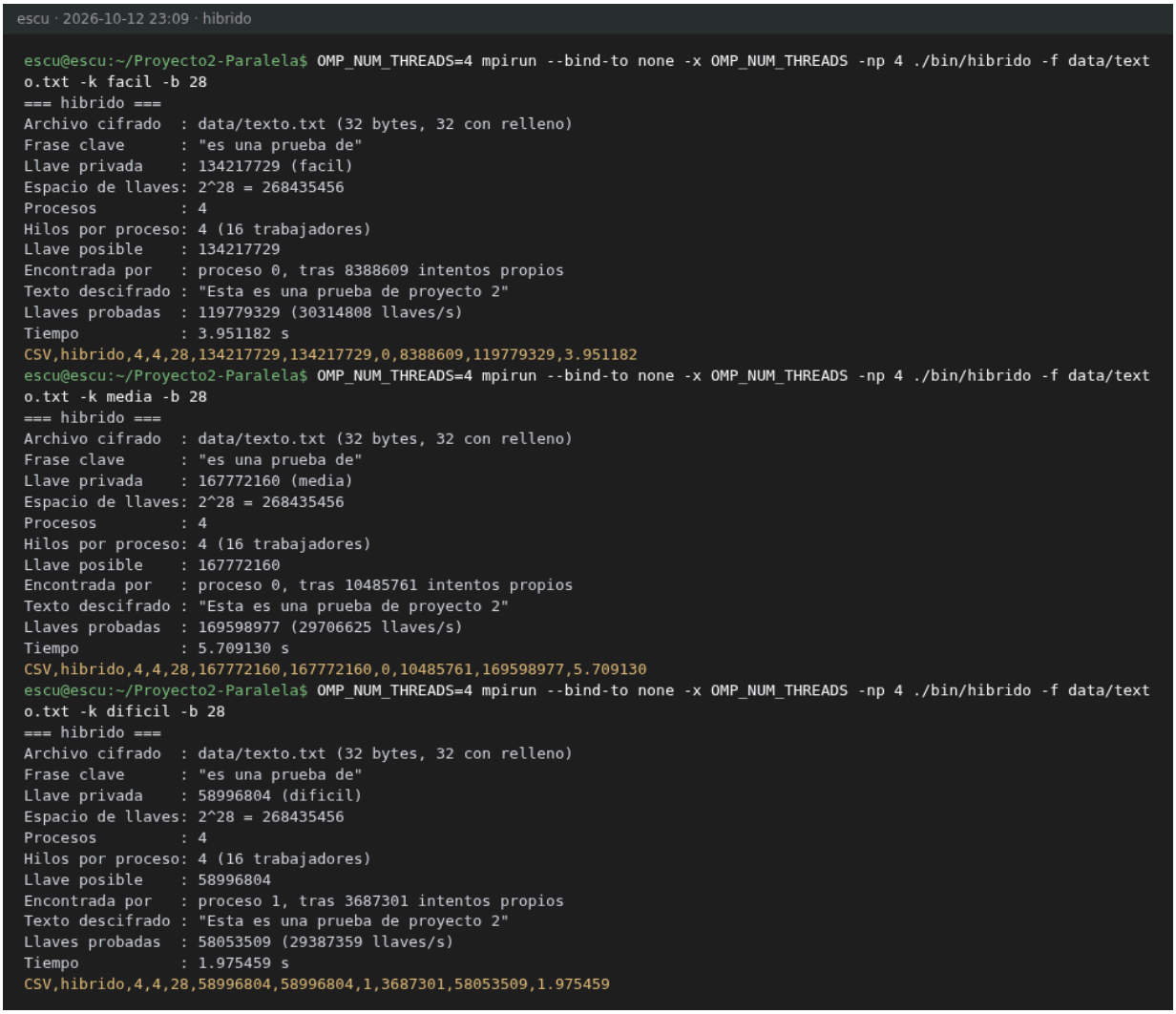


Figura 22. Híbrido con 4 procesos y 4 hilos por proceso.

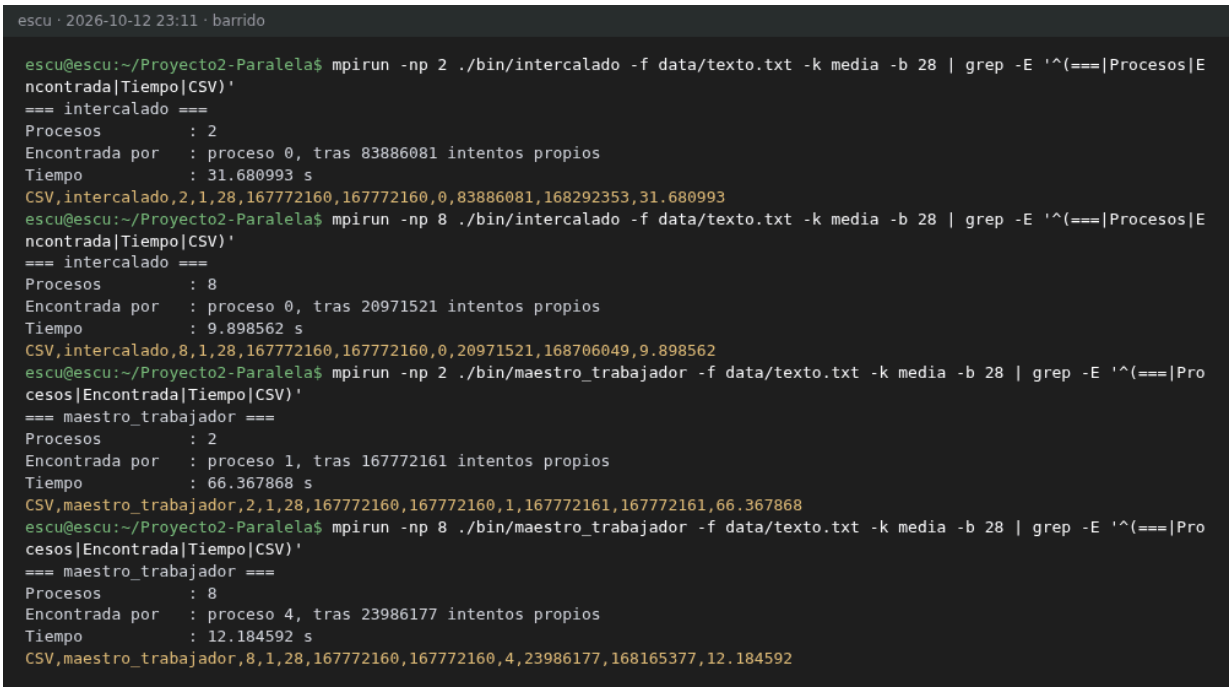


Figura 23. Barrido de procesos: intercalado y maestro-trabajador con 2 y 8 procesos. Solo se muestran las líneas clave de cada reporte.